

Algebraic Attack on Convolutional Neural Network with Max Pooling

Zirui Chen¹, Shi Tang², Zhengchao Gao², Yongjia Su², Lingyue Qin¹, and
Xiaoyang Dong¹

¹ Tsinghua University, Beijing, People's Republic of China

chenzr25@mails.tsinghua.edu.cn

qinly@tsinghua.edu.cn

xiaoyangdong@tsinghua.edu.cn

² Shandong University, Qingdao, People's Republic of China

shi.tang@mail.sdu.edu.cn

chao.qwq@mail.sdu.edu.cn

yongjia.su@mail.sdu.edu.cn

Abstract. Recovering the weights and biases of deep neural networks (DNNs) via black-box input-output queries - known as parameter extraction attacks - has been extensively studied for ReLU-based fully connected neural networks (FCNNs), but remains unexplored for Convolutional Neural Networks (CNNs) with max pooling function, a core architecture for computer vision and multimedia processing. The key challenge lies in CNN's max pooling layer, which introduces an additional nonlinearity and hides ReLU critical points, rendering existing FCNN extraction methods inapplicable. To address this gap, we propose the first cryptanalytic extraction attack tailored for CNNs with the max pooling function.

First, we establish an algebraic representation of CNNs, formally proving that CNNs are piecewise linear functions - enabling the extension of linearity-based extraction principles. We then identify two novel types of critical points in CNNs: (1) ReLU-Pooling Critical Points (RPCPs), where a ReLU neuron is at its zero-input critical point and its output is selected by max pooling; and (2) Pooling Switching Points (PSPs), where two neurons within a local receptive field yield identical maximum outputs, triggering a switch in the pooling selection.

Leveraging these critical points, we design complementary extraction techniques: a pattern matching method for RPCPs to recover partial signatures and signs (exploiting the property that unselected pooling neurons have negative outputs), and an internal differential extraction attack for PSPs - inspired by cryptographic internal differential analysis - to recover high-accuracy signatures. Given that PSPs are far more abundant and efficient than RPCPs (verified by experiments), and RPCPs are capable of bias recovery, we integrate both methods: PSPs enable efficient signature extraction, while a single RPCP recovers the sign and bias.

We evaluate our attack on multiple CNN architectures, including the LeNet-5 with modern architectures, trained on random data, MNIST, and CIFAR-10. Experimental results demonstrate that our approach

achieves high extraction accuracy using polynomial query complexity and runtime, even for deep CNN layers. This work fills a research gap in CNN security.

Keywords: Convolutional Neural Networks · Model Extraction

1 Introduction

Deep Neural Networks (DNNs) computes a function from inputs to outputs, and the architecture and weights and biases of the network determine the function that is expressed. Treating DNN as a black box, and knowing its inputs and outputs, how to extract weights and biases has been a long-standing problem, dating back as far as 30 years ago [2,12]. At 2005, Lowd and Meek proposed an adversarial reverse engineering attack in [29]. Recently, new ideas from research teams in industry and academia have continued to emerge regarding this problem [1,23,31,35,39]. Targeting the DNNs with alternating fully connected linear layers and Rectified Linear Units (ReLU) activation (denoted as *ReLU-based fully connected neural network (FCNN)*), at CRYPTO 2020, Carlini, Jagielski, and Mironov achieved a breakthrough with a *cryptographic differential attack* [7], which requires querying polynomial number of the raw outputs of FCNN to extract the unsigned weights. While, the sign recovery is a brute-force guessing method, which makes the overall time complexity of Carlini *et al.*’s algorithm exponential [7]. At EUROCRYPT 2024, Canales-Martínez *et al.* [5] introduced the *neuron wiggle technique* to recover the neuron signs in polynomial time complexity. At NIPS 2024, Foerster *et al.* [13] built end-to-end attack on practical models by combining the methods of Carlini *et al.* [7] and Canales-Martínez *et al.* [5]. Liu *et al.* [28] introduced more techniques to extract parameters in deep layers. At ASIACRYPT 2025, Chen *et al.* [9] studied the parameter extraction on FCNN with Parametric Rectified Linear Unit (PReLU) function [18].

In another direction, still targeting *ReLU-based FCNN*, but where the attackers can only access the final classification labels (*e.g.*, “dog” or “car”), *i.e.*, the *hard-label scenario*, Chen *et al.* [8] first proposed an extraction method requiring a polynomial number of queries but an exponential execution time. At EUROCRYPT 2025, Carlini *et al.* [6] introduced an extraction attack in hard-label setting with polynomial number of queries and polynomial time by analyzing the geometric shape of its decision boundaries. Later, Canales-Martínez [4] proposed to recover the output layer, where no ReLU functions exist. In 2025, Ito, Miura, and Todo found that as the depth of the attack-target grows, the attack hypothesis of Carlini *et al.* [6] may become impractical, and they proposed a *cross-layer extraction* to solve this problem. Till now, all the cryptanalytic extraction attacks focus on the FCNN, where the ReLU-like active functions are the only non-linear factor in the network. In this paper, we consider the cryptanalytic extraction attacks on the *Convolutional Neural Network (CNN)* with max pooling function for the first time.

1.1 Convolutional Neural Network and the Challenges in the Model Extraction.

The concept of convolutional neural networks (CNNs) can be traced back to the concept of receptive fields in the 1960s. In 1980s, Fukushima *et al.* [15] proposed the concept of neurocognitive machine. In 1998, LeCun officially proposed convolutional neural networks and designed the well-known LeNet-5 [26] network for handwritten digit recognition. CNN has been one of the most popular and modern neural network, acting critical roles on processing of images [25], video [40], series signals [41], text data [42], and the like. Well-known architectures of CNN include LeNet-5 [26], AlexNet [25], VGG [36], GoogLeNet [38], ResNet [19], DenesNet [20] etc.

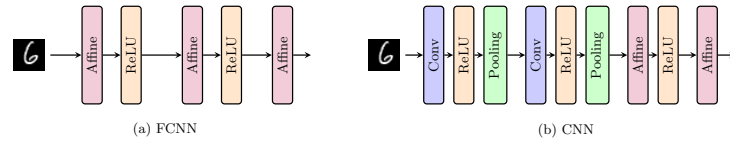


Fig. 1: FCNN and CNN

In Figure 1, the key difference between CNN and ReLU-based FCNN is that CNN includes an Pooling layer (note that the convolutional layer can be transformed into a linear affine layer in Sect. 2.2). In the pooling layer, a pooling function (usually use the max or average function) is used to compress the outputs of all the neurons in a *Local Receptive Field*, and output single value. Typically, the Max pooling function is commonly used in practice due to its unique advantages in feature extraction, anti-interference ability, and prevention of overfitting, which have been verified and applied in many classic CNN papers [26,3,25,36,30]. However, when Max pooling function³ is applied, there will be another nonlinear function in the network besides the ReLU function, which brings two problems when applying the Carlini *et al.*'s parameter extraction attacks.

- The first problem is that the critical point of the ReLU function will be hidden by max pooling in CNN. By varying the inputs in tiny vicinity of the critical point, it is expected that the network's output will change nonlinearly as a result of this ReLU transition. However, for CNN network, the outputs of the neurons will be selected by a max pooling function, and only the maximum output of the neurons in a *Local Receptive Field* will be selected as the output of the pooling layer. There is a significant probability that the output (whose value is around 0) of the neuron at the critical point will not

³In fact, different from Max pooling, the Average pooling does not introduce additional nonlinearity to the network, and Carlini *et al.*'s technique [7] can be trivially applied to this case.

be selected, then the network’s output will change linearly though a neuron is at its critical point. This case happens easily when one of the outputs of the other neurons within the same *Local Receptive Field* is greater than 0.

- The second problem is that the max pooling layer introduces an additional nonlinearity, and a similar critical point can be spotted when two outputs of the neurons within a *Local Receptive Field* are the two maximum values (e.g., $a = b$). By varying the inputs in tiny vicinity of the critical point, a will exceed b , or b will exceed a . Then, the max pooling layer’s selection will switch between a and b . Obviously, this kind of critical point is quite different from Carlini *et al.*’s critical point. How to leverage this new critical point to extract parameters will be a new problem.

1.2 Our Contributions

We introduce an algebraic representation of the CNN, and transform all the states and operations into vectors and matrices, and therefore, formally make it clear that CNN is also piecewise linear function, *i.e.*, within a tiny linear neighborhood of input X , CNN acts as a linear function, and all operations in a CNN are matrix-vector multiplications. We formally introduce two kinds of critical points, *i.e.*,

- **ReLU-Pooling Critical Point (RPCP):** It is the point where a neuron is at the critical point (input=0), and the subsequent max pooling layers select its output, and having a non-linear effect on the CNN’s output. Based on RPCPs, Carlini *et al.*’s differential attack is applied to recover the partial signatures (some entries are impossible to recover with a single RPCP, since the ReLU functions suppress negative output [7, Section 4.3.2]). We propose a new *pattern matching method* for CNN to merge multiple partial signatures to get the full one. Again more RPCPs are needed to improve accuracy of the signature by applying Carlini *et al.*’s least squares method. Interestingly, the sign recovery is quite easy based on a property of RPCP, *i.e.*, the unselected neurons’ outputs by the max pooling function should be negative.
- **Pooling Switching Point (PSP):** It is the point where the outputs of two neurons within a Local Receptive Field of pooling achieve the same maximum value. To leverage the PSP, we invent a new parameter extraction technique inspired by the cryptographic internal differential attack [33,10], named as *internal differential extraction* attack, where the propagation of the difference between different parts of a single Pooling Switching Point X are studied through CNN operations.

Advantages and limitations of RPCP and PSP methods:

- According to our experiments, RPCPs are relatively rare in CNN, since the pooling selection hides the output (whose value is around 0) of the neuron at the critical point with significant probability. Therefore, merging partial signatures or improving the accuracy of signatures that require many RPCPs are difficult.

- + RPCP method can easily recover the sign and bias based on its property.
- + According to our experiments, PSPs are more common in CNNs than RPCP. Based on the *internal differential extraction* attack, the signature (without bias) can be extracted in higher accuracy with shorter runtime and fewer model queries than RPCP methods.
- PSP method can not recover the bias.

Unified Model Combining PSP and RPCP. Taking the above advantages and limitations, we integrate the PSP and RPCP methods as a unified parameter extraction model, where PSP is used to recover the signature (without bias) and sign, while only a single (or a few) RPCP is needed to recover the bias. Besides, we introduce a method named *targeted heuristic search strategy for RPCP*, to tangle the case that even one RPCP is hard to find. This method leverages the signature recovered by PSP to directly detect the points close to a RPCP.

End-to-end Experiments. Practical parameter extractions on several CNNs are performed, including the popular LeNet-5 with modern architectures. Taken the full $(m + n)$ -deep CNN (with m convolutional rounds and n FCNN rounds) as a black box, with the inputs and the raw outputs of the black box, two kinds of experiments are given:

- We recover all the parameters of the m convolutional rounds of the first 5 CNNs (including the modern LeNet-5) of Table 1, where $(m+n)$ means only the parameters of the m convolutional rounds are recovered.
- We give an experiment of recovering the full parameters of a $(m + n)$ -deep CNN, which is a reduced version of LeNet-5 with 2 standard Convolutional rounds, 1 reduced FCNN round (the modern LeNet-5 has 2 FCNN rounds), and an output layer (*i.e.*, 3 hidden layers). To our knowledge, this matches the depth of the deepest architecture - 3 hidden layers [7], among existing end-to-end black-box full-model parameter extraction works.

We also attack the standard modern LeNet-5 (specified in Sect. 7.1), extracting the full convolutional block and 93.33% weights of FCNN Round 1 (not full due to dead neurons and persistent neurons according to [22]).

The results and the comparison with Carlini *et al.*'s experiment [7] and Foster *et al.*'s experiment [13] are given in the last four rows of Table 1.

All the source codes and experiments will be made available soon in:

XXX

Comparison with [37]. At ACISP 2026, Sun *et al.* [37] proposed a crypt-analytic extraction of convolutional neural networks with **average pooling**. However, on the commonly used max pooling function, they claimed that [37, Section 5]:

“However, the non-linear nature of max pooling layers, which only retains the maximum value within each region, inherently leads to significant information loss and makes the operation irreversible. As a result,

reconstructing the original input or parameters from these layers presents considerable difficulties, making the handling of such layers a persistent challenge for future research.”

This paper addresses the challenges posed by the max-pooling function and, for the first time, recovers the parameters of a CNN employing max pooling layers.

Table 1: Experiments on different CNNs. Convolution kernel dimensions are denoted as $(c_{in}^{(k)}, c_{out}^{(k)}, h_c^{(k)}, w_c^{(k)})$, with stride $s_c^{(k)} = 1$. Pooling kernels are 2×2 with stride $s_p^{(k)} = 2$. The accuracy terms $(\varepsilon, 0)$ is defined in Sect. 2.3, and $\max|\theta - \hat{\theta}|$ directly measures the maximum extraction error of model parameters.

Models ($m+n$)	Trainset	Architecture [†] $d^{(k)}-d_f^{(k)}-d^{(k+1)}$	Kernel	Queries	$(\varepsilon, 0)$	$\max \theta - \hat{\theta} $
(1+1)	Random	C_1 : 64–36–9	(1,1,3,3)	$2^{9.98}$	$2^{-43.19}$	$2^{-47.52}$
(2+1)	MNIST	C_1 : 1024–784–196 C_2 : 196–100–25	(1,1,5,5) (1,1,5,5)	$2^{15.06}$	$2^{-30.20}$	$2^{-34.65}$
(2+2)*	MNIST	C_1 : 1024–4704–1176 C_2 : 1176–1600–400	(1,6,5,5) (6,16,5,5)	$2^{27.29}$	$2^{-23.38}$	$2^{-27.31}$
(3+1) ^{p2}	MNIST	C_1 : 1024–1024–256 C_2 : 256–256–64 C_3 : 64–64–16	(1,1,5,5) (1,1,5,5) (1,1,5,5)	$2^{16.90}$	$2^{-27.54}$	$2^{-31.15}$
(2+2) ^{p1}	CIFAR10	C_1 : 3072–3072–768 C_2 : 768–1024–256	(3,3,3,3) (3,4,3,3)	$2^{18.37}$	$2^{-29.13}$	$2^{-36.19}$
(2+1) [‡]	MNIST	C_1 : 1024–4704–1176 C_2 : 1176–1600–400 F: 400–20–10	(1,6,5,5) (6,16,5,5) –	$2^{27.65}$	$2^{-22.06}$	$2^{-27.14}$
(2+2)* [♣]	MNIST	C₁ : 1024–4704–1176 C₂ : 1176–1600–400 F: 400–120 –84–10	(1,6,5,5) (6,16,5,5) –	$2^{27.31}$	–	$2^{-26.03}$
(0+3) [7]	MNIST	F: 40–20–10–10–1	–	$2^{17.8}$	$2^{-23.4}$	$2^{-27.1}$
(0+3) [13] [♠]	MNIST	F: 784–16– 16–16 –1	–	$2^{25.4}$	–	–

[†]: C_i indicates Convolutional Round i ; F denotes FCNN block with $d^{(1)} - \dots - d^{(n+1)}$.

^{*}: LeNet-5 modern model, specified in in Sect. 7.1.

^{p1, p2}: p_i indicates $pad^{(k)} = i$ defined as “same padding” in Supplementary Material B; otherwise, $pad^{(k)} = 0$.

[‡]: Reduced version of LeNet-5 with only one FCNN Round.

[♣]: Red bold layer in Architecture indicates end-to-end extracting the full convolutional block and 93.33% weights of FCNN Round 1 (not full due to existing dead neurons and persistent neurons according to [22]).

[♠]: Red bold layer in Architecture indicates extracting the single layer, with prior layers’ extractions assumed to be correct.

2 Attack Overview

2.1 Basic Definitions and Notations

- **FCNN**: Fully Connected Neural Network (FCNN) in Figure 1.
- **CNN**: Convolutional Neural Network (CNN) in Figure 1.
- **ReLU**: Rectified Linear Unit.
- **FCNN Round**: Including a linear function $f^{(k)}$ called fully-connected layer, and a nonlinear function σ (component-wise ReLU function).
- **Convolutional Round**: Including Convolutional layer $f_c^{(k)}$, nonlinear activation layer $\sigma_c^{(k)}$ (ReLU), and pooling layer $\rho^{(k)}$ (Max Pooling).
- **(m + n)-Deep Convolutional Neural Network (CNN)**: CNN with m convolutional rounds and n FCNN rounds, as well as an output layer with a fully connected linear layer.
- $(\mathbf{c}_{in}^{(k)}, \mathbf{c}_{out}^{(k)}, \mathbf{h}_c^{(k)}, \mathbf{w}_c^{(k)})$: Convolution layer with $\mathbf{c}_{in}^{(k)}$ input and $\mathbf{c}_{out}^{(k)}$ output channels, and the size of kernel matrices are $h_c^{(k)} \times w_c^{(k)}$.
- $\mathbf{X}^{(k)}, \mathbf{Y}^{(k)}, \mathbf{Z}^{(k)}$: Input vectors of the operations $f_c^{(k)}, \sigma_c^{(k)}, \rho^{(k)}$ in the k -th convolutional rounds, $k \geq 1$. $X^{(k)} \in \mathbb{R}^{d^{(k)}}$, $Y^{(k)}, Z^{(k)} \in \mathbb{R}^{d_f^{(k)}}$, since $\sigma_c^{(k)}$ does not change the dimension. $X^{(k)}[i]$ is the i -th ($0 \leq i < d^{(k)}$) entry of $X^{(k)}$.
- $\mathbf{A}^{(k)}$: Matrix whose entry in i -th row and j -th is $A_{i,j}^{(k)}$, i, j starts from 0.
- **RPCP, PSP, FCP**: ReLU-Pooling Critical Point, Pooling Switching Point, Critical points in the Fully-connected Block, respectively.
- **LRF-C**, Local Receptive Field of Convolutional Layer: A sub-matrix of $IN^{(k)}$, which is multiplied by *Convolution Kernel Matrix* $C^{(k)}$ to produce each entry in $Y^{(k)}$. The LRF-C will be different for different element of $Y^{(k)}$. Let $Y^{(k)}[t]$ corresponds to the t -th ($0 \leq t \leq d_f^{(k)} - 1$) LRF-C.
- **LRF-P**, Local Receptive Field of Pooling Layer: A sub-matrix of $Z^{(k)}$, which is processed by a pooling function to produce the each entry of $X^{(k+1)}$. The LRF-P will be different for different element of $X^{(k+1)}$. Let $X^{(k+1)}[t]$ corresponds to the t -th ($0 \leq t \leq d^{(k+1)} - 1$) LRF-P.

2.2 Algebraic View of CNN

Previous works [7,5,8,6,9] concentrate on the Fully Connected Neural Network (FCNN) in Figure 1, which is composed of a sequence of functions alternating between linear functions $f^{(k)} : \mathbb{R}^{d^{(k)}} \mapsto \mathbb{R}^{d^{(k+1)}}$ ($k \geq 1$, called fully-connected layers), and a nonlinear function σ (component-wise ReLU function):

$$\mathcal{F} = f^{(r+1)} \circ \sigma \circ f^{(r)} \circ \sigma \circ \dots \circ f^{(2)} \circ \sigma \circ f^{(1)}. \quad (1)$$

In Figure 1, a typical CNN architecture consists of an initial sequence of convolutional, nonlinear ReLU, and pooling layers (this paper focuses on the Max Pooling), followed by a series of fully-connected layers, which are referred as the *Convolutional Block* and the *Fully-connected Block*, respectively.

Definition 1. (($m+n$)-Deep Convolutional Neural Network) A ($m+n$)-deep Convolutional Neural Network (CNN) is a function $\mathcal{F}_\theta$ parameterized by θ that takes inputs from an input space $\mathbb{R}^{d^{(1)}}$ and returns values in an output space $\mathbb{R}^{d^{(m+n+2)}}$. The function $\mathcal{F}_\theta : \mathbb{R}^{d^{(1)}} \rightarrow \mathbb{R}^{d^{(m+n+2)}}$ is composed of two sequential blocks: the first block (Convolutional Block) consists of m layers of alternating convolutional layers $f_c^{(k)}$, nonlinear activation layers $\sigma_c^{(k)}$, and pooling layers $\rho^{(k)}$ ($1 \leq k \leq m$); the second block (Fully-connected Block) consists of n layers of alternating linear layers $f^{(k)}$ and activation functions $\sigma^{(k)}$:

$$\mathcal{F}_\theta = \underbrace{f^{(n+1)} \circ \sigma^{(n)} \circ \dots \circ \sigma^{(1)} \circ f^{(1)}}_{\text{Fully-connected Block}} \circ \underbrace{\rho^{(m)} \circ \sigma_c^{(m)} \circ f_c^{(m)} \circ \dots \circ \rho^{(1)} \circ \sigma_c^{(1)} \circ f_c^{(1)}}_{\text{Convolutional Block}}. \quad (2)$$

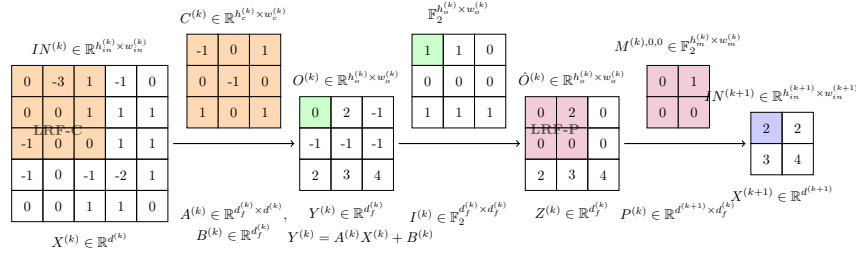


Fig. 2: Symbolic View of CNN

Convolutional Layers. The CNN's inputs are formulated as matrices. The convolution operation of the convolutional Layer $f_c^{(k)}$ employs a sliding window mechanism, where a small matrix of learnable parameters (the *Convolution Kernel Matrix*) traverses the input from left to right, top to bottom. At each position, the sum of element-wise products between the *Convolution Kernel Matrix* and the overlapping local input patch (*Local Receptive Field of convolutional layer*, abbreviated as *LRF-C*), is computed to generate a single output value. A shared bias is then added to the result at each position to constitute the output matrix (*Feature Map*) of the convolutional layer.

In the algebraic view, the input matrix of $f_c^{(k)}$ is denoted as $IN^{(k)}$ with dimension $h_{in}^{(k)} \times w_{in}^{(k)}$, so $d^{(k)} = h_{in}^{(k)} \cdot w_{in}^{(k)}$. The convolution kernel matrix is defined by the shape $(c_{in}^{(k)}, c_{out}^{(k)}, h_c^{(k)}, w_c^{(k)})$, where $c_{in}^{(k)}$ and $c_{out}^{(k)}$ denote the number of input and output channels, and $h_c^{(k)}, w_c^{(k)}$ denote the height and width of the convolution kernel matrix $C^{(k)}$, respectively.

For brevity, we will first consider the simple case of single input and output channel, i.e., $c_{in}^{(k)} = c_{out}^{(k)} = 1$, and the case of multiple channels will be analyzed

in Section 7. Given a convolution stride $s_c^{(k)}$, the $h_o^{(k)} \times w_o^{(k)}$ output matrix or the *feature map* of $f_c^{(k)}$ is denoted as $O^{(k)}$, where

$$h_o^{(k)} = \left\lfloor \frac{h_{in}^{(k)} - h_c^{(k)}}{s_c^{(k)}} \right\rfloor + 1, \quad w_o^{(k)} = \left\lfloor \frac{w_{in}^{(k)} - w_c^{(k)}}{s_c^{(k)}} \right\rfloor + 1. \quad (3)$$

For briefness, we let the convolution stride $s_c^{(k)} = 1$. The input of $f_c^{(k)}$ can be a $h_{in}^{(k)} \times w_{in}^{(k)}$ $IN^{(k)}$ or a $d^{(k)}$ -dimension vector $X^{(k)}$. Denote $IN^{(k)} = [a_{i,j}^{(k)}]_{h_{in}^{(k)} \times w_{in}^{(k)}}$ and the i -th ($0 \leq i \leq h_{in}^{(k)} - 1$) row $IN_i^{(k)} = (a_{i,0}^{(k)}, a_{i,1}^{(k)}, \dots, a_{i,w_{in}^{(k)}-1}^{(k)})$. Then the vector $X^{(k)} = (IN_0^{(k)}, IN_1^{(k)}, \dots, IN_{h_{in}^{(k)}-1}^{(k)})^\top$.

Denote the kernel matrix $C^{(k)} = [c_{i,j}^{(k)}]_{h_c^{(k)} \times w_c^{(k)}}$, where $C_i^{(k)} = (c_{i,0}^{(k)}, c_{i,1}^{(k)}, \dots, c_{i,w_c^{(k)}-1}^{(k)})$. Then convolution operation computes the output matrix $O^{(k)}$ as

$$O_{i,j}^{(k)} = \sum_{p=0}^{h_c^{(k)}-1} \sum_{q=0}^{w_c^{(k)}-1} IN_{i+p,j+q}^{(k)} C_{p,q}^{(k)} + b^{(k)}, \quad (4)$$

where the bias $b^{(k)}$ is the same for all the outputs. In vector form, the output matrix $O^{(k)}$ will be a $(d_f^{(k)} = h_o^{(k)} \cdot w_o^{(k)})$ -dimension vector $Y^{(k)} = (O_0^{(k)}, \dots, O_{h_o^{(k)}-1}^{(k)})^\top$.

Definition 2. (Convolutional Matrix) Let $c_{in}^{(k)} = c_{out}^{(k)} = s_c^{(k)} = 1$. In the k -th convolutional layer, given a convolution kernel matrix $C^{(k)}$, the function $f_c^{(k)} : \mathbb{R}^{d^{(k)}} \rightarrow \mathbb{R}^{d_f^{(k)}}$, is defined as $f_c^{(k)}(X^{(k)}) = A^{(k)}X^{(k)} + B^{(k)}$, where $X^{(k)} \in \mathbb{R}^{d^{(k)}}$ is the input vector. $A^{(k)} = (A_0^{(k)\top}, A_1^{(k)\top}, \dots, A_{d_f^{(k)}-1}^{(k)\top})^\top \in \mathbb{R}^{d_f^{(k)} \times d^{(k)}}$, whose i -th row ($0 \leq i \leq d_f^{(k)} - 1$) is:

$$A_i^{(k)} = (\underbrace{0, \dots, 0}_{w_{in}^{(k)} \cdot \left\lfloor \frac{i}{w_o^{(k)}} \right\rfloor}, \underbrace{0, \dots, 0, C_0^{(k)}, 0, \dots, 0}_{i \bmod w_o^{(k)}}, \dots, \underbrace{0, \dots, 0, C_{h_c^{(k)}-1}^{(k)}, 0, \dots, 0}_{i \bmod w_o^{(k)}}, \underbrace{0, \dots, 0}_{padding}), \quad (5)$$

and $B^{(k)} = (b^{(k)}, b^{(k)}, \dots, b^{(k)})^\top \in \mathbb{R}^{d_f^{(k)}}$ (the bias for each neuron is the same).

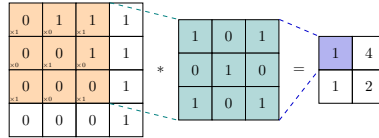


Fig. 3: Example of Convolution Operation (bias $b^{(k)}=0$)

Consider a concrete scenario with $c_{in}^{(k)} = c_{out}^{(k)} = s_c^{(k)} = 1$ as shown in Figure 3, where the input matrix $IN^{(k)}$ has dimensions $h_{in}^{(k)} \times w_{in}^{(k)} = 4 \times 4$, i.e., $d^{(k)} = 16$. Given a $h_c^{(k)} \times w_c^{(k)} = 3 \times 3$ kernel matrix $C^{(k)} = [c_{i,j}^{(k)}]_{3 \times 3}$, the output matrix $O^{(k)}$ is of dimension $h_o^{(k)} \times w_o^{(k)} = 2 \times 2$, i.e., $d_f^{(k)} = 4$. According to Def. 2,

$$A^{(k)} = \begin{pmatrix} c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 & 0 & 0 & 0 & 0 \\ 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 \\ 0 & 0 & 0 & 0 & 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} \end{pmatrix}.$$

Nonlinear Activation Layer. The k -th activation layer $\sigma_c^{(k)}$ of a CNN is by a set of parallel $d_f^{(k)}$ nonlinear activation functions $\text{ReLU}(x_i) = \max\{x_i, 0\}$ for $i = 0, \dots, d_f^{(k)} - 1$. The input of $\sigma_c^{(k)}$ layer is the $h_o^{(k)} \times w_o^{(k)}$ matrix $O^{(k)}$, and denote its output as $\hat{O}^{(k)}$ with the same dimension. In vector form, the input is denoted as $Y^{(k)} = (y_0, y_1, \dots, y_{d_f^{(k)}-1})^\top$, then $\sigma_c^{(k)}$ layer can be expressed as a matrix $I^{(k)}$ multiplied by $Y^{(k)}$, i.e., $Z^{(k)} = (\hat{O}_0^{(k)}, \dots, \hat{O}_{h_o^{(k)}-1}^{(k)})^\top = \sigma_c^{(k)}(Y) = I^{(k)}Y^{(k)}$, where

$$I^{(k)} = \begin{pmatrix} \tau_0^{(k)} & 0 & \dots & 0 \\ 0 & \tau_1^{(k)} & \dots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & \tau_{d_f^{(k)}-1}^{(k)} \end{pmatrix}, \quad 0 \leq i \leq d_f^{(k)} - 1, \quad \tau_i^{(k)} = \begin{cases} 1, & \text{if } y_i > 0, \\ 0, & \text{if } y_i \leq 0. \end{cases} \quad (6)$$

Pooling Layers. In pooling layer, a small, fixed-size window (analogous to a convolution kernel) pooling kernel slides across a feature map (input matrix $\hat{O}^{(k)}$) with a predefined stride. The region of the input matrix covered by the pooling window is also referred to as the *Local Receptive Field* of Pooling (LRF-P for short). For each window position, it applies an pooling function (usually use the max or average function) to compress the contents in the LRF-P into a single value, producing a dimension reduced output matrix (denoted as $IN^{(k+1)}$). Unlike convolution kernels, pooling kernels are parameter-free (no learnable weights and biases) — their behavior is determined solely by size, stride, and compressing rule. Usually, the Max pooling function is more widely used than the average pooling in practice due to its unique advantages in feature extraction, anti-interference ability, and prevention of overfitting, which have been verified and applied in many classic CNN papers [26,3,25,36,30]. Therefore, this paper focuses on the Max pooling. Algebraically, we reformulate the operation of the Pooling layer as follows.

In the pooling layer $\rho^{(k)}$, assuming the Max pooling kernel is applied to a LRF-P of size $h_m^{(k)} \times w_m^{(k)}$ with stride $s_\rho^{(k)}$. The input is $h_o^{(k)} \times w_o^{(k)}$ matrix $\hat{O}^{(k)}$

in Figure 2. The output is denoted as a $h_\rho^{(k)} \times w_\rho^{(k)}$ matrix $IN^{(k+1)}$, where:

$$h_\rho^{(k)} = \left\lfloor \frac{h_o^{(k)} - h_m^{(k)}}{s_\rho^{(k)}} \right\rfloor + 1, \quad w_\rho^{(k)} = \left\lfloor \frac{w_o^{(k)} - w_m^{(k)}}{s_\rho^{(k)}} \right\rfloor + 1.$$

In particular, when the pooling kernel size is $h_m^{(k)} \times w_m^{(k)} = 2 \times 2$ and the stride is $s_\rho^{(k)} = 2$, and the windows partition the input into disjoint LRF-Ps. The output size calculation reduces to:

$$h_\rho^{(k)} = \left\lfloor \frac{h_o^{(k)}}{2} \right\rfloor, \quad w_\rho^{(k)} = \left\lfloor \frac{w_o^{(k)}}{2} \right\rfloor.$$

The entry of the i -th row and j -th column ($0 \leq i \leq h_\rho^{(k)} - 1$, $0 \leq j \leq w_\rho^{(k)} - 1$) of the output matrix $IN^{(k+1)}$ takes the maximum value of the elements in the $(i \cdot w_\rho^{(k)} + j)$ -th LRF-P of $\hat{O}^{(k)}$, whose indexes are denoted as $\Omega_{i,j} = \{(i \cdot s_\rho^{(k)} + \bar{h}, j \cdot s_\rho^{(k)} + \bar{w}), 0 \leq \bar{h} \leq h_m^{(k)} - 1, 0 \leq \bar{w} \leq w_m^{(k)} - 1\}$. Then,

$$IN_{i,j}^{(k+1)} = \max\{\hat{O}_{\Omega_{i,j}}^{(k)}\}. \quad (7)$$

When the input matrix $\hat{O}^{(k)}$ is given, and the max pooling operation can be expressed as multiplying a series of Boolean matrices $M^{(k),i,j} \in \mathbb{F}_2^{h_m^{(k)} \times w_m^{(k)}}$ ($0 \leq i \leq h_\rho^{(k)} - 1$, $0 \leq j \leq w_\rho^{(k)} - 1$) by different $h_m^{(k)} \times w_m^{(k)}$ LRF-P of $\hat{O}^{(k)}$,

$$IN_{i,j}^{(k+1)} = \sum_{0 \leq h \leq h_m^{(k)} - 1, 0 \leq w \leq w_m^{(k)} - 1} M_{\{h,w\}}^{(k),i,j} \cdot \hat{O}_{\{i \cdot s_\rho^{(k)} + h, j \cdot s_\rho^{(k)} + w\}}^{(k)}, \quad (8)$$

where

$$M_{\{h,w\}}^{(k),i,j} = \begin{cases} 1, & \text{if } \hat{O}_{\{i \cdot s_\rho^{(k)} + h, j \cdot s_\rho^{(k)} + w\}}^{(k)} = \max\{\hat{O}_{\Omega_{i,j}}^{(k)}\}, \\ 0, & \text{else.} \end{cases} \quad (9)$$

In vector form, the input and output vector of pooling layer $\rho^{(k)}$ are $Z^{(k)} = (\hat{O}_0^{(k)}, \dots, \hat{O}_{h_o^{(k)}-1}^{(k)})^\top \in \mathbb{R}^{d_f^{(k)} = h_o^{(k)} \cdot w_o^{(k)}}$ and $X^{(k+1)} = (IN_0^{(k+1)}, IN_1^{(k+1)}, \dots, IN_{h_\rho^{(k)}-1}^{(k+1)})^\top \in \mathbb{R}^{d^{(k+1)}}$, $d^{(k+1)} = h_\rho^{(k)} \cdot w_\rho^{(k)}$.

Definition 3. (Boolean Pooling Matrix) *There exists a $d^{(k+1)} \times d_f^{(k)}$ Boolean pooling matrix $P^{(k)}$ so that $\rho^{(k)} : \mathbb{R}^{d_f^{(k)}} \rightarrow \mathbb{R}^{d^{(k+1)}}$ satisfying $X^{(k+1)} = \rho^{(k)}(Z^{(k)}) = P^{(k)} \cdot Z^{(k)}$. Assuming the LRF-Ps are disjoint and square (i.e., $s_\rho^{(k)} = w_m^{(k)} = h_m^{(k)}$), then the max pooling operation on the r -th LRF-P can be defined as the r -th row ($0 \leq r \leq d^{(k+1)} - 1$) of $P^{(k)}$ is*

$$P_r^{(k)} = (\underbrace{0, \dots, 0}_{w_o^{(k)} \cdot i \cdot s_\rho^{(k)}}, \underbrace{0, \dots, 0}_{j \cdot s_\rho^{(k)}}, \underbrace{M_0^{(k),i,j}, \dots, 0}_{padding}, \dots, \underbrace{0, \dots, 0}_{j \cdot s_\rho^{(k)}}, \underbrace{M_{h_m^{(k)}-1}^{(k),i,j}, \dots, 0}_{padding}, \underbrace{0, \dots, 0}_{padding}), \quad (10)$$

where $i = \left\lfloor \frac{r}{w_\rho^{(k)}} \right\rfloor$, $j = r \bmod w_\rho^{(k)}$.

Obviously, similar to the matrix of $I^{(k)}$ for $\sigma_c^{(k)}$ layer, the pooling matrix $P^{(k)}$ varies with the input $X^{(1)}$ since Eq. (9). For example, the input matrix $\hat{O}^{(k)}$ has dimensions $h_o^{(k)} \times w_o^{(k)} = 4 \times 4$, i.e., $d_f^{(k)} = 16$, the Max pooling kernel is of $h_m^{(k)} \times w_m^{(k)} = 2 \times 2$ with stride $s_\rho^{(k)} = 2$. The output matrix $IN^{(k+1)}$ will have dimension $h_\rho^{(k)} \times w_\rho^{(k)} = 2 \times 2$, i.e., $d^{(k+1)} = 4$. The $d^{(k+1)} \times d_f^{(k)} = 4 \times 16$ pooling matrix $P^{(k)} = \begin{pmatrix} P_{0,0}^{(k)} & \mathbf{0}_{2 \times 8} \\ \mathbf{0}_{2 \times 8} & P_{1,1}^{(k)} \end{pmatrix}$, where $\mathbf{0}_{2 \times 8}$ is 2×8 zero matrix, and

$$P_{0,0}^{(k)} = \begin{pmatrix} M_{0,0}^{(k),0,0} & M_{0,1}^{(k),0,0} & 0 & 0 & M_{1,0}^{(k),0,0} & M_{1,1}^{(k),0,0} & 0 & 0 \\ 0 & 0 & M_{0,0}^{(k),0,1} & M_{0,1}^{(k),0,1} & 0 & 0 & M_{1,0}^{(k),0,1} & M_{1,1}^{(k),0,1} \end{pmatrix},$$

$$P_{1,1}^{(k)} = \begin{pmatrix} M_{0,0}^{(k),1,0} & M_{0,1}^{(k),1,0} & 0 & 0 & M_{1,0}^{(k),1,0} & M_{1,1}^{(k),1,0} & 0 & 0 \\ 0 & 0 & M_{0,0}^{(k),1,1} & M_{0,1}^{(k),1,1} & 0 & 0 & M_{1,0}^{(k),1,1} & M_{1,1}^{(k),1,1} \end{pmatrix}.$$

Definition 4. (Fully-connected Layer) The k -th fully-connected layer within the fully-connected block of the CNN is a function $f^{(k)} : \mathbb{R}^{d^{(k)}} \rightarrow \mathbb{R}^{d^{(k+1)}}$ given by an affine transformation:

$$f^{(k)}(X) = A^{(k)}X^{(k)} + B^{(k)},$$

where $X^{(k)} \in \mathbb{R}^{d^{(k)}}$ represents the input vector. The weight matrix $A^{(k)} \in \mathbb{R}^{d^{(k+1)} \times d^{(k)}}$ and the bias vector $B^{(k)} \in \mathbb{R}^{d^{(k+1)}}$ are composed of floating-point numbers.

The matrix for the nonlinear layer σ (ReLU functions) of the fully-connected block can be similarly defined by Eq. (6).

Definition 5. (Layer Merging) While we focus on the convolutional block of the $(m+n)$ network, assuming we fully know the first $k-1$ layers of convolutional block, and we are currently recovering layer k . Let F_x^{k-1} and G_x^{k+1} represent, respectively, the fully recovered and non-recovered part of the CNN.

$$\mathcal{F} = \underbrace{f^{(n+1)} \circ \dots \circ \sigma^{(1)} \circ f^{(1)} \circ \rho^{(m)} \circ \dots \circ f_c^{(k+1)}}_{G_x^{k+1}} \circ \rho^{(k)} \circ \sigma_c^{(k)} \circ f_c^{(k)} \circ \underbrace{\rho^{(k-1)} \circ \dots \circ f_c^{(1)}}_{F_x^{k-1}}$$

If we restrict inputs $X^{(1)}$ to be in a tiny linear neighborhood of X , G_x^{k+1} and F_x^{k-1} become $G_x^{k+1}(X^{(k+1)}) = G^{k+1}X^{(k+1)} + B^{k+1}$ and $F_x^{k-1}(X) = F^{k-1}X^{(1)} + B^{k-1}$, where the matrices $F^{k-1} \in \mathbb{R}^{d^{(k)} \times d^{(1)}}$, $B^{k-1} \in \mathbb{R}^{d^{(k)}}$, $G^{k+1} \in \mathbb{R}^{d^{(n+2)} \times d^{(k+1)}}$, $B^{k+1} \in \mathbb{R}^{d^{(n+2)}}$, respectively.

Definition 6. (Model Parameters) The parameters θ of a $(m+n)$ -deep neural network f_θ are the concrete assignments to the weights $C^{(k)}$, biases $B^{(k)}$ for $k \in 1, \dots, m$ in Convolutional Block and the concrete assignments to weights $A^{(k)}$ and $B^{(k)}$ for $k \in 1, \dots, n+1$ in Fully-connected Block.

Definition 7. (Neuron State [7]) Let $\mathcal{V}(\eta; X)$ denote the value that neuron η takes with $X \in \mathbb{R}^{d^{(1)}}$ before applying its corresponding activation function σ . If $\mathcal{V}(\eta; X) > 0$ (respectively, $\mathcal{V}(\eta; X) < 0$), the neuron state of η is activated (respectively, inactivated). If $\mathcal{V}(\eta; X) = 0$, the neuron state is critical.

2.3 Adversarial Goals and Assumptions

The adversarial goal is to extract all secret parameters given in Def. 6 of CNN with its input and the output. We make the following assumptions about Oracle and the capabilities of the attacker:

- **Architecture knowledge.** We require knowledge of the model architecture of the neural network, including dimensions of kernel matrix, max pooling kernel, and all the strides $s_c^{(k)}, s_p^{(k)}$.
- **Full-domain inputs.** The attacker can query arbitrary inputs from $\mathbb{R}^{d^{(1)}}$.
- **Precise Computation.** The CNN is specified and evaluated using a sufficiently high precision floating-point arithmetic.
- **ReLU Activations.** All activation functions $\sigma^{(k)}$ are the ReLU function.
- **Raw Output Accessible.** Given input X , the raw output of the CNN is accessible to the adversary. The same assumption is also used by Carlini *et al.*'s attack [7].

$(\varepsilon, 0)$ -Functional Equivalence [7]. To evaluate the precision of our attack, we adopted the concept of (ε, ξ) -Functional Equivalence proposed by Carlini *et al.* [7] and subsequently used by Chen *et al.* [8,9].

Definition 8. $((\varepsilon, \xi)$ -Functional Equivalence [7]) Two neural networks f_θ and $f_{\hat{\theta}}$ are (ε, ξ) -functional equivalent on the input dataset S if

$$Pr_{x \in S}(|f_\theta(x) - f_{\hat{\theta}}(x)| \leq \varepsilon) \geq 1 - \xi.$$

In our analysis, we specifically focus on the $(\varepsilon, 0)$ -functional equivalence given by Carlini *et al.* [7] which represents an upper bound on the maximum error of the output. We adopt the error bounds propagation method introduced by Carlini *et al.* [7, Section 6.2] to compute the $(\varepsilon, 0)$ -functional equivalence.

3 Input Space Partition and Critical Points Recognition

The input space $\mathbb{R}^{d^{(1)}}$ is partitioned into many linear regions. Within each distinct linear region, the activation states of all neurons and the selection indices of the pooling layers remain constant, the neural network computes a fixed linear function over the region. However, when linearly varying inputs crosses the boundaries between these regions, the output exhibits nonlinear changes. According to the definitions in Section 2.2, it is evident that both convolutional and fully connected layers represent linear operations. The nonlinearity is jointly introduced by Activation Layer and Pooling Layer:

- **ReLU Activation.** This determines whether a neuron’s input is propagated to the subsequent layer, where a nonlinear transition between continuous zero state (inactivated) and linear non-zero state (activated) occurs. For a fixed input, the activation matrix is deterministic.
In Carlini *et al.*’s seminal attack [7] and all the follow-up results [5,6,8,9] are based on the nonlinearity introduced by ReLU, which is called *critical point*. The adversary studies on the behavior of the FCNN in the vicinity of critical points, which are defined as inputs in whose tiny vicinity exactly one ReLU input in one of the layers changes sign. It is expected the FCNN’s output will change nonlinearly as a result of this ReLU transition, which makes this event noticeable when we observe the network’s outputs by varying the inputs in those tiny vicinity.
It is important to note that for this critical hyperplane to be effective, the neuron must be selected by the subsequent pooling layers; otherwise, its activation state has no impact on the final output.
- **Max Pooling.** This operation selects the maximum value within each local receptive field. Since different neurons have different gradients in their linear outputs, a shift in the pooling selection results in a nonlinear change in the final output. Similar to the activation layer, the pooling matrix is fixed when the input is determined.

3.1 Does Carlini *et al.*’s Critical Point Work For CNN?

At EUROCRYPT 2024, Canales-Martinez *et al.* [5] stated that the attack based on Carlini *et al.*’s critical point also works for CNN without modification:

“Our attack can be applied without modifications to convolutional networks since they can be described as a special form of a fully connected network.”

In FCNN, by changing the inputs in a tiny vicinity of the critical point, it can be identified by testing whether the network’s output changes non-linearly. However, unlike FCNN, the non-linearity of CNN is introduced through ReLU or Max pooling, which may lead to two problem when applying Carlini *et al.*’s critical point methods.

Problem I: Taking Figure 4 shows an example. The critical point $\bar{X}^{(1)}$ causes the 0-th entry of the input vector $Y^{(k)}$ of $\sigma_c^{(k)}$ layer to be zero. However, the 2×2 Max Pooling layer does not select the output of $Y^{(k)}[0]$. It selects the maximum value among the neuron $Y^{(k)}[0, 1, 3, 4]$, which outputs the value 2 in Figure 4.

Therefore, changing the inputs within a tiny vicinity of the critical point $\bar{X}^{(1)}$ will cause the network’s output to change linearly, making the critical point $\bar{X}^{(1)}$ undetectable. Only when the Max Pooling layer selects the output of neuron α , the network’s output will be affected nonlinearly by changing the inputs around $\bar{X}^{(1)}$. As shown in Figure 5, the values in neuron $Y^{(k)}[0, 1, 3, 4]$ are 0, -2, -1, -1, and neuron α is at the critical point. The Max Pooling layer is expected to

select the output of neuron α when changing the inputs near $\bar{X}^{(1)}$, and cause the network's output to change non-linearly, which makes the critical point $\bar{X}^{(1)}$ detectable. Formally, the critical point $\bar{X}^{(1)}$ makes $Y_t^{(k)} = 0$ ($0 \leq t \leq d_f^{(k)} - 1$) of

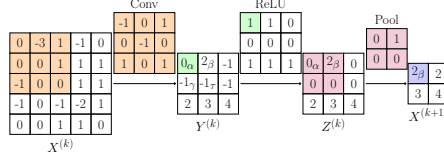


Fig. 4: Undetectable Critical Point

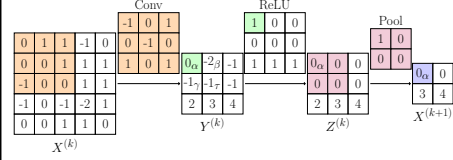


Fig. 5: Detectable Critical Point

$\sigma_c^{(k)}$. According to Eq. (6), $\tau_t = 0$. Let $\bar{X}_+^{(1)} = \bar{X}^{(1)} + \epsilon$ and $\bar{X}_-^{(1)} = \bar{X}^{(1)} - \epsilon$ with small enough ϵ , and assuming the corresponding $Y_{t+}^{(k)} > 0$ and $Y_{t-}^{(k)} < 0$. Hence, the corresponding $\tau_{t+} = 1$, $\tau_{t-} = 0$. Considering the matrix $P^{(k)} \cdot I^{(k)}$, in order to apply Carlini *et al.*'s attack, the entries $\tau_{t+} = 1$, $\tau_{t-} = 0$ should result in different matrix $P^{(k)} \cdot I^{(k)}$, else the two inputs $\bar{X}_+^{(1)}$ and $\bar{X}_-^{(1)}$ will have a linear impact on the output of the CNN. Note that the entries of $P^{(k)} \cdot I^{(k)}$ that related to τ_t is the t -th column of $P^{(k)} \cdot I^{(k)}$. Only when $(P_{0,t}^{(k)}, P_{1,t}^{(k)}, \dots, P_{d^{(k+1)}-1,t}^{(k)})^T \neq \mathbf{0}$, there exists i , so that $P_{i,t}^{(k)} = 1$. Then, $(P^{(k)} \cdot I^{(k)})_{i,t} = \tau_t$, which makes the $P^{(k)} \cdot I^{(k)}$ different for $\tau_{t+} = 1$ and $\tau_{t-} = 0$. Moreover, in the following layers' operations, τ_t should also appear in the operations matrix to affect the CNN's output, *i.e.*, the matrix $G^{k+1} \cdot P^{(k)} \cdot I^{(k)}$ should also be affected by τ_t , otherwise, the critical point is undetectable either. We give a definition on this kind of detectable critical point.

Definition 9. (ReLU-Pooling Critical Point, RPCP) A ReLU-Pooling Critical Point (RPCP) is an input $X^{(1)} \in \mathbb{R}^{d^{(1)}}$ that makes the input of ReLU function in $\sigma_c^{(k)}$ layer 0, and by varying the inputs in the tiny vicinity of $X^{(1)}$, the output of CNN changes nonlinearly.

Property 1. If $X^{(1)}$ is a RPCP corresponding to the t -th neuron in layer k , then $A_t^{(k)} X^{(k)} + B_t^{(k)} = 0$, the inputs of other neurons in the same Local Reception Field are smaller than 0.

Problem II: Besides the RPCP, the network's output changes non-linearly may also because the Pooling layer selection changes. Look at Figure 6, in layer k , there is no 0 in $Y^{(k)}$, *i.e.*, no RPCP. However, in $Z^{(k)}$, the input $X^{(1)}$ makes $Z_{(k)}[0] = Z_{(k)}[1] > Z_{(k)}[3] = Z_{(k)}[4]$, hence the max pooling kernel matrix $M^{(k),0,0} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$. Let $\bar{X}_+^{(1)} = \bar{X}^{(1)} + \epsilon$ and $\bar{X}_-^{(1)} = \bar{X}^{(1)} - \epsilon$ with small enough ϵ , and assuming the corresponding $(Z_+^{(k)}[0] < 2, Z_+^{(k)}[1] > 2)$ and $(Z_-^{(k)}[0] > 2,$

$Z_-^{(k)}[1] < 2$). Then, the selection matrix $M^{(k),0,0}$ for $\bar{X}_+^{(1)}$ and $\bar{X}_-^{(1)}$ will be different. Thus, the output of CNN will nonlinearly impacted.

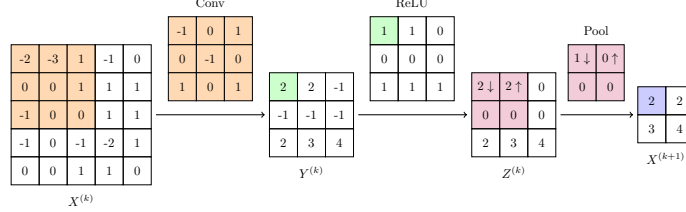


Fig. 6: Pooling Switch Point (PSP)

Definition 10. (Pooling Switching Point, PSP) A Pooling Switching Point (PSP) is an input $X^{(1)} \in \mathbb{R}^{d^{(1)}}$ such that two inputs within a Local Receptive Field achieve the same maximum value. By varying the inputs in the tiny vicinity of $X^{(1)}$, the output of CNN changes nonlinearly.

Property 2. If $X^{(1)}$ is a PSP corresponding to the i -th and j -th neuron in layer k , then $A_i^{(k)} X^{(k)} + b_i^{(k)} = A_j^{(k)} X^{(k)} + b_j^{(k)}$, and the inputs of other neurons in the same LRF-P are smaller than it.

Thus, nonlinearity in CNNs is jointly induced by the changes in neuron activation states via ReLU and the shifts in neuron selection via pooling operations. Meanwhile, note that in the Fully-connected Block, critical points of each neuron making corresponding ReLU changed still exist.

Definition 11. (Fully-connected Critical Point, FCP [7]) A Fully-connected Critical Point (FCP) is an input $X^{(1)} \in \mathbb{R}^{d^{(1)}}$ that makes the input of ReLU function $\sigma^{(k)}$ of a neuron in the Fully-connected Block 0.

For instance, if $X^{(1)}$ is a FCP corresponding to the i -th neuron in layer k in the Fully-connected Block, then $A_i^{(k)} X^{(k)} + b_i^{(k)} = 0$.

We collect RPCPs, PSPs and FCPs from all layers by continuously varying inputs along any straight lines in $\mathbb{R}^{d^{(1)}}$, and searching for the breakpoints where nonlinear changes happen.

Remarks. In the Fully-connected Block, the FCPs will definitely affect the output. However, the Convolutional Block is in fact a series of partial connected layers. Therefore, the RPCPs and PSPs should satisfy the properties 1 and 2 respectively, and they should also be selected by subsequent convolutional rounds (which must be activated and selected through subsequent activation and pooling operations until the model's output) to affect the final output of CNN.

4 Parameter Extraction with ReLU-Pooling Critical Point

4.1 Signature Extraction: Algebraic View of Carlini *et al.*'s differential attack

The signature extraction is mainly based on Carlini *et al.*'s differential attack [7]. We formalize it in an algebraic view. Consider a specific input $X \in \mathbb{R}^{d^{(1)}}$ identified as a ReLU-Pooling Critical Point (RPCP)⁴ corresponding to the i -th neuron in the k -th convolutional layer. A small perturbation $\vec{\delta}$ is introduced such that the points $X + \vec{\delta}$ and $X - \vec{\delta}$ remain within the local linear neighborhood of X . The activation states of the neurons of the network change exclusively at the i -th neuron of layer k . Consequently, the corresponding activation matrices, denoted as $I_+^{(k)}$ (for $X + \vec{\delta}$) and $I_-^{(k)}$ (for $X - \vec{\delta}$), differ only at the i -th diagonal entry. Specifically, the i -th entry is 1 (assuming $X + \vec{\delta}$ is at the activated state) and 0 (assuming $X - \vec{\delta}$ is at the inactivated state).

Based on the difference propagation properties of the network, within same linear region, the output differences can be expressed as:

$$\mathcal{F}(X + \vec{\delta}) - \mathcal{F}(X) = G^{k+1} P^{(k)} I_+^{(k)} A^{(k)} F^{k-1} \vec{\delta}, \quad (11)$$

$$\mathcal{F}(X) - \mathcal{F}(X - \vec{\delta}) = G^{k+1} P^{(k)} I_-^{(k)} A^{(k)} F^{k-1} \vec{\delta}. \quad (12)$$

Define the second-order differential function as $\mathcal{H}(X; \vec{\delta})$ by the subtraction of (11) and (12), *i.e.*, $\mathcal{H}(X; \vec{\delta}) = \mathcal{F}(X + \vec{\delta}) + \mathcal{F}(X - \vec{\delta}) - 2\mathcal{F}(X)$, therefore

$$\mathcal{H}(X; \vec{\delta}) = G^{k+1} P^{(k)} (I_+^{(k)} - I_-^{(k)}) A^{(k)} F^{k-1} \vec{\delta}, \quad (13)$$

where the term $I_+^{(k)} - I_-^{(k)}$ yields a diagonal matrix with a single non-zero entry (value 1) at the entry (i, i) , effectively acting as a selection operator. The product $P^{(k)} (I_+^{(k)} - I_-^{(k)})$ extracts the specific column of the pooling matrix $P^{(k)}$ associated with the critical neuron i . Since X is a RPCP, the output of the neuron i is the unique maximum within its LRF-P, implying that the corresponding entry of $P^{(k)}$ is 1 (*i.e.*, it is selected), while the contributions from other neurons in the window are zero.

Assuming the LRF-Ps are disjoint and the r -th ($0 \leq r < d^{(k+1)}$) LRF-P covers the critical neuron i . Assuming the network has a single output (targeting one of the components of the output vector). The matrix G^{k+1} reduces to a single row vector, denoted as $g = (g_0, g_1, \dots, g_{d^{(k+1)}-1})$, then

$$G^{k+1} P^{(k)} (I_+^{(k)} - I_-^{(k)}) A^{(k)} = (g_r A_{i,0}^{(k)}, g_r A_{i,1}^{(k)}, \dots, g_r A_{i,d^{(k)}-1}^{(k)}). \quad (14)$$

⁴The Step 3 in Sect. 6 discusses how to distinguish between RPCP, PSP, and FCP.

By sampling n different directions $\vec{\delta}^{(j)}$ ($0 \leq j \leq n-1$) and querying the model to get $\mathcal{H}(X; \vec{\delta}^{(j)})$, we construct a system of equations

$$\begin{pmatrix} (F^{k-1}\vec{\delta}^{(0)})^\top \\ (F^{k-1}\vec{\delta}^{(1)})^\top \\ \vdots \\ (F^{k-1}\vec{\delta}^{(n-1)})^\top \end{pmatrix} \begin{pmatrix} g_r A_{i,0}^{(k)} \\ g_r A_{i,1}^{(k)} \\ \vdots \\ g_r A_{i,d^{(k)}-1}^{(k)} \end{pmatrix} = \begin{pmatrix} \mathcal{H}(X; \vec{\delta}^{(0)}) \\ \mathcal{H}(X; \vec{\delta}^{(1)}) \\ \vdots \\ \mathcal{H}(X; \vec{\delta}^{(n-1)}) \end{pmatrix} = \vec{\mathcal{H}} \quad (15)$$

Denote $S = (\vec{\delta}^{(0)}, \vec{\delta}^{(1)}, \dots, \vec{\delta}^{(n-1)})^\top$, the system of equations can be rewritten as:

$$S F^{k-1 \top} g_r A_i^{(k) \top} = \vec{\mathcal{H}}. \quad (16)$$

4.2 Pattern Matching Method: Recover Full Signature with Multiple RPCPs

To ensure the linear system above possesses a unique solution, the coefficient matrix of Eq (16) should maintain full column rank, i.e., $\text{rank}(S(F^{k-1})^\top) = d^{(k)}$. As discovered by Carlini *et al.* [7, Section 4.3.2] in the attack on FCNN, the rank of the matrix is usually not full since the ReLU functions suppress negative output, and therefore some weights are impossible to recovered (denoted as NaN values) with only one critical point. They have to find at least two critical points for the same i -th neuron to derive several partial signatures and merge them to extract the full signature [7,28].

Similarly, given one RPCP, the $\text{rank}(S(F^{k-1})^\top)$ of Eq (16) in CNNs is also affected by ReLU functions (i.e., the rank of $I^{(k-1)}$ may not full), leading to $\text{rank}(S(F^{k-1})^\top) < d^{(k)}$. Note that according to Definitions 2 and 3, the convolutional matrix $A^{(k)}$ and the pooling matrix $P^{(k)}$ are full column rank. Therefore, some entries of $g_r A_i^{(k) \top}$ are impossible to recover with only one RPCP. Similar to Carlini *et al.*'s method [7], we also need multiple RPCPs to recover several partial signatures and merge them, since $\text{rank}(S(F^{k-1})^\top) < d^{(k)}$ in most cases.

The difference is that our RPCPs do not necessarily correspond to the same neuron, since according to Eq. (5), each row of $A^{(k)}$ contains the full information about the kernel matrix $C^{(k)}$, whose entries are to be recovered. According to Eq. (5), different RPCP for different neuron in layer k recover different rows of $A^{(k)}$. *The problem is to determine which rows of $A^{(k)}$ the RPCPs recover, and then merge them according to Eq. (5) to get the full signature.* In theory, given a row $\hat{A}_i^{(k)}$ recovered by a RPCP, the row number i could also be determined by the non-zero and zero positions in the Eq. (5). However, in practice, both the zero-values and kernel weights (nonzero) in $A_i^{(k)}$ have possibility to be recovered as NaN. Consequently, it can not be directly recognize which row the RPCP recovers, i.e., one can not directly determine which neuron is associated with the RPCP. Therefore, it is not trivial to align the partial recovered $C^{(k)}$ and merge them. We propose the *Pattern Matching Method* to solve this problem.

Pattern Matching Method. According to Eq. (5), for the i -th row of $A^{(k)}$, define the corresponding i -th convolution kernel pattern vector $\vec{V}_i \in \mathbb{F}_2^{d^{(k)}}$ as

$$\vec{V}_i[j] = \begin{cases} 1, & \text{if } A_i^{(k)}[j] \neq 0, \\ 0, & \text{else.} \end{cases} \quad (17)$$

Define its bitwise inverse vector as $\vec{\bar{V}}_i \in \mathbb{F}_2^{d^{(k)}}$ with $\vec{\bar{V}}_i[j] = \vec{V}_i[j] \oplus 1$.

Suppose we have an RPCP corresponding to the i -th ReLU function of layer k , denote the computed vector from Eq. (16) as $\hat{A}_i^{(k)} \approx g_r A_i^{(k)}$, where there exists NaN values. According to Eq. (18), the recovered $\hat{A}_i^{(k)}$ could be

$$\hat{A}_i^{(k)} = (\underbrace{\mathbf{0}}_{w_{in}^{(k)} \cdot \bar{i}}, \underbrace{\mathbf{0}}_{\bar{i}}, \underbrace{\hat{C}_{0,0}^{(k)}, \text{NaN}, \dots, \hat{C}_{0, w_c^{(k)}-1}^{(k)}}_{w_{in}^{(k)}}, \underbrace{\mathbf{0}_{pad}, \dots, \mathbf{0}_{\bar{i}}, \text{NaN}, \text{NaN}, \dots, \hat{C}_{h_c^{(k)}-1, w_c^{(k)}-1}^{(k)}}_{w_{in}^{(k)}}, \underbrace{\mathbf{0}_{pad}, \mathbf{0}_{pad}}_{pad}), \quad (18)$$

where $\bar{i} = \lfloor \frac{i}{w_o^{(k)}} \rfloor$, $\bar{i} = i \bmod w_o^{(k)}$. Please note that any entry in $\hat{A}_i^{(k)}$ may contain a NaN value, including those entries whose actual value should be 0. Then, we define two Binary mask vectors for $\hat{A}_i^{(k)}$ as $\vec{\alpha}, \vec{\alpha}' \in \mathbb{F}_2^{d^{(k)}}$, where

$$\vec{\alpha}[j] = \begin{cases} 1, & \text{if } A_i^{(k)}[j] \neq 0 \text{ or } A_i^{(k)}[j] = \text{NaN} \\ 0, & \text{else,} \end{cases}, \quad \vec{\alpha}'[j] = \begin{cases} 1, & \text{if } A_i^{(k)}[j] = 0 \text{ or } A_i^{(k)}[j] = \text{NaN} \\ 0, & \text{else.} \end{cases}$$

Given an RPCP, the $\vec{\alpha}, \vec{\alpha}' \in \mathbb{F}_2^{d^{(k)}}$ are computed. Then, we use the following *matching principle* to detect which ReLU function the RPCP corresponds to. Suppose the RPCP corresponds to the i -th ReLU function, then we consider the i -th convolution kernel pattern vector $\vec{V}_i$ and the extracted vector $\hat{A}_i^{(k)}$.

1. Non-zero Consistency Principle: Obviously, when $\vec{V}_i[j] = 1$, the $\hat{A}_i^{(k)}[j]$ will be either non-zero or NaN, i.e., $\vec{V}_i[j] = \vec{\alpha}[j]$ and

$$\vec{V}_i \cdot \vec{\alpha}^T = w_c^{(k)} \cdot h_c^{(k)}. \quad (19)$$

2. Zero Consistency Principle: Similarly, When $\vec{\bar{V}}_i[j] = 1$, $\vec{\alpha}'[j] = 1$. we have

$$\vec{\bar{V}}_i \cdot \vec{\alpha}'^T = d^{(k)} - w_c^{(k)} \cdot h_c^{(k)}. \quad (20)$$

We iteratively compute Eq. (19) and (20) for each $0 \leq i \leq d_f^{(k)} - 1$ and check if these two equation holds. If Eq. (19) and (20) holds, we deduce that the RPCP is corresponding to the i -th ReLU function, and then extract partial values of the $\hat{C}^{(k)}$ from Eq. (18). For multiple RPCPs, recover the partial $\hat{C}^{(k)}$ and merge them to build a full $\hat{C}^{(k)}$.

Remarks. By employing the pattern matching method, we identify the specific neuron and determine the constituent elements of the kernel matrix from a single RPCP. This makes the reconstruction of kernel weights simple. The identification mechanism is both intuitive and robust: results derived from noisy points (non-RPCPs) fail to find the kernel pattern vector $\vec{V}_i$ that confirms to the matching principle, causing the pattern matching algorithm to inherently reject them.

Bias Recovery. In layer k , after the neuron i at the RPCP is identified, the bias (note that the bias for each neuron is the same according to Definition 2) is computed, since the input the neuron i is zero:

$$\hat{b}^{(k)} = -\hat{A}_i^{(k)}(F^{k-1}x + B^{k-1}) . \quad (21)$$

4.3 Sign Recovery

After recover $\hat{A}^{(k)}$ and $\hat{B}^{(k)}$ without sign, we compute the input vector of the neurons as $\hat{A}^{(k)}(F^{k-1}x + B^{k-1}) + \hat{B}^{(k)}$, where x is the RPCP of the i -th neuron of layer k . Assuming the neuron i belongs to the t -th LRF-P Ω_t . According to Prop. 1, all the inputs of the neurons within Ω_t should be ≤ 0 . Then,

- if all these inputs are ≤ 0 , the sign of the extracted kernel matrix is correct.
- if there exists positive input, the extracted kernel matrix has an inverted sign, and multiply the matrices $\hat{A}^{(k)}$ and $\hat{B}^{(k)}$ by “ -1 ”.

5 Parameter Extraction with Pooling Switching Point

The attack based on RPCP given in Section 4 relies highly on the successful acquisition of RPCPs. When extending the attack to deeper layers, the proportion of valid RPCPs among all discovered critical points diminishes greatly and the efficiency of the attack decreases significantly (detail experiments confirmed this phenomenon in Sect. 6). Therefore, we propose an more efficient parameter extraction attack based on the Pooling Switching Point (PSP). To leverage the PSP, a novel *internal differential attack* on CNN is applied, instead of Carlini *et al.*’s differential attack [7].

5.1 Internal Differential Extraction Attack on CNN

Internal differential attack [33] is proposed by Peyrin at CRYPTO 2010 against Grøstl, which is subsequently applied to the cryptanalysis of Keccak by Dinur, Dunkelman, and Shamir [10]. This section proposes the first internal differential attack on CNN, named as the *internal differential extraction* attack.

According to Property 2 of the Pooling Switching Point (PSP): Given a PSP X , in layer k , suppose the i -th and j -th neuron’s input within the same t -th LRF-P Ω_t ($0 \leq t < d^{(k+1)}$) have the same largest value, which satisfies:

$$Y^{(k)}[i] = A_i^{(k)}(F^{(k-1)}X + B^{k-1}) + b^{(k)} = Y^{(k)}[j] = A_j^{(k)}(F^{(k-1)}X + B^{k-1}) + b^{(k)} > 0. \quad (22)$$

Then, the pooling selection switches between neuron i and neuron j for any input within the X ’s local linear neighborhood. It is important to note that $A_i^{(k)}$ is a sparse vector according to Eq. (5), which only contains non-zero entries at the indices where the convolution kernel is applied. This sparsity allows

us to explicitly extract the positional indices of the convolution kernel's receptive field (LRF-C). Denote the vector form of kernel matrix $C^{(k)}$ as $\vec{C}^{(k)} = (C_0^{(k)}, C_1^{(k)}, \dots, C_{h_c^{(k)}-1}^{(k)})^\top$. So the length of the vector $\vec{C}^{(k)}$ is $l_c^{(k)} = h_c^{(k)} w_c^{(k)}$. Assuming the stride $s_c^{(k)} = 1$, and let $\mathcal{K}_i^{(k)} \subset \{0, 1, \dots, d^{(k)} - 1\}$ denote the set of indices belonging to the LRF-C of the i -th ($0 \leq i \leq d_f^{(k)} - 1$) neuron, then,

$$\mathcal{K}_i := \left\{ w_{in}^{(k)} \cdot \left\lfloor \frac{i}{w_o^{(k)}} \right\rfloor + i \bmod w_o^{(k)} + w_{in}^{(k)} \cdot \left\lfloor \frac{t}{w_c^{(k)}} \right\rfloor + t \bmod w_c^{(k)}, 0 \leq t < l_c \right\}. \quad (23)$$

The set $\mathcal{K}_i^{(k)}$ picks out the nonzero entries of $A_i^{(k)}$ according to Eq. (5). Then,

$$A_i^{(k)} X^{(k)} = \vec{C}^{(k)} \cdot X^{(k)}[\mathcal{K}_i]. \quad (24)$$

Similar equation can be deduced for neuron j . Then, subtract the two equations, according to Eq. (22), we get

$$\vec{C}^{(k)} \cdot (X^{(k)}[\mathcal{K}_i] - X^{(k)}[\mathcal{K}_j]) = 0. \quad (25)$$

This is quite similar to the internal-differential attack [33], where the difference between two distinct parts of the unique input $X^{(k)}$ are studied through the cryptographic/CNN operations. The input difference is $\nabla X^{(k)} = X^{(k)}[\mathcal{K}_i] - X^{(k)}[\mathcal{K}_j]$, and the output difference after the convolutional layer is 0, which will be identified as a PSP. Given a PSP, Eq. (25) is used to construct the system of equations to compute $\vec{C}^{(k)}$. With the deduced parameters of $k - 1$ layers, $X^{(k)} = F^{k-1}X + B^{k-1}$ is deduced. The only unknown are the positions of the pooling selection switch, *i.e.*, the i and j .

5.2 Extracting i and j of the Pooling Selection Switch

Similar to Carlini *et al.*'s differential attack [7], we introduce a small perturbation $\vec{\delta}$ such that the points $X + \vec{\delta}$ and $X - \vec{\delta}$ remain within the local linear neighborhood of X . Exactly one of the two points will make the pooling selects the i -th neuron's output, and assuming the point is $X + \vec{\delta}$, where the corresponding pooling matrix is denoted as $P_+^{(k)}$. The other point will make j -th neuron's output selected, and $P_-^{(k)}$ is defined similarly. Then, we have

$$\mathcal{F}(X + \vec{\delta}) - \mathcal{F}(X) = G^{k+1} P_+^{(k)} I^{(k)} A^{(k)} F^{k-1} \vec{\delta}, \quad (26)$$

$$\mathcal{F}(X) - \mathcal{F}(X - \vec{\delta}) = G^{k+1} P_-^{(k)} I^{(k)} A^{(k)} F^{k-1} \vec{\delta}. \quad (27)$$

Assuming the t -th ($0 \leq t \leq d^{(k+1)} - 1$) LRF-P includes the neuron i and j ($i < j$) whose input values are $Y^{(k)}[i]$ and $Y^{(k)}[j]$, and the matrices $P_+^{(k)}$ and $P_-^{(k)}$ will be different only in their t -th row. Specifically, in $P_+^{(k)}$, this row is $(0 \cdots 0, 1_i, 0 \cdots 0, 0_j, 0 \cdots 0)$, while in $P_-^{(k)}$, the same row becomes $(0 \cdots 0, 0_i, 0 \cdots 0, 1_j, 0 \cdots 0)$.

Still, the second-order difference function, denoted as $\mathcal{H}(x; \delta) = \mathcal{F}(X + \vec{\delta}) + \mathcal{F}(X - \vec{\delta}) - 2\mathcal{F}(X)$:

$$\mathcal{H}(X; \vec{\delta}) = G^{k+1}(P_+^{(k)} - P_-^{(k)})I^{(k)}A^{(k)}F^{k-1}\vec{\delta}. \quad (28)$$

Similar to the RPCP method in Section 4, when the matrix G^{k+1} is reduced to a single row vector $g = (g_0, g_1, \dots, g_{d^{(k)}-1})$, then

$$G^{k+1}(P_+^{(k)} - P_-^{(k)})I^{(k)}A^{(k)} = g_t(A_i^{(k)} - A_j^{(k)}), \quad (29)$$

where t -th row of $P_+^{(k)} - P_-^{(k)}$ is $(0 \cdots 0, 1_i, 0 \cdots 0, -1_j, 0 \cdots 0)$, and the other rows are zero vectors. By sampling n distinct directions $\vec{\delta}^{(\ell)}$ ($0 \leq \ell \leq n-1$), we can construct a system of equations and recover $g_t(A_i^{(k)} - A_j^{(k)})$,

$$g_t(A_i^{(k)} - A_j^{(k)}) \cdot F^{k-1}\vec{\delta}^{(\ell)} = \mathcal{H}(X; \vec{\delta}^{(\ell)}). \quad (30)$$

According to Eq. (5), $A_i^{(k)}$ and $A_j^{(k)}$ correspond to i -th and j -th row vectors of the matrix $A^{(k)}$, where their non-zero values are offset from each other. Assuming $i < j$, we deduce i and j from the positions of the first and last nonzero entry of the recovered vector $g_t(A_i^{(k)} - A_j^{(k)})$, respectively, according to Eq. (5). Then $\mathcal{K}_i$ and $\mathcal{K}_j$ are deduced from (23).

Handling vector containing NaN. Sometimes the calculated row vector $\Delta_{ij}\hat{A}^{(k)} \approx g_t(A_i^{(k)} - A_j^{(k)})$ from Eq. (30) is incomplete with similar reason to Section 4.2. It is necessary to check whether the deduced $\hat{i}$ and $\hat{j}$ matches the practical $g_t(A_i^{(k)} - A_j^{(k)})$, following the matching principles:

1. Union Non-zero Consistency: At indices where either $A_i^{(k)}$ or $A_j^{(k)}$ is non-zero, the corresponding element in the difference vector $\Delta_{ij}\hat{A}^{(k)}$ should be either non-zero or NaN.
2. Common Zero Consistency: At indices where both $A_i^{(k)}$ and $A_j^{(k)}$ are zero, the corresponding element in $\Delta_{ij}\hat{A}^{(k)}$ should be either 0 or NaN.

Following the above two principles and Eq. (5), we determine $\hat{i}$ by finding the first $w_c^{(k)}$ consecutive non-zero or NaN entries that contains at least one non-zero value in $\Delta_{ij}\hat{A}^{(k)}$ (denote this partial vector as $\vec{\beta}$), then

- If the first element $\vec{\beta}[0]$ is not NaN, return the position index of $\vec{\beta}[0]$ as $\hat{i}$.
- If the $\vec{\beta}[0]$ is NaN, let $\hat{i}$ choose two possible values, *i.e.*,
 - $\hat{i}$ is the index of $\vec{\beta}[0]$ in Δ_{ij} by assuming the true value $\vec{\beta}[0]$ is not 0.
 - Incrementing the index of $\vec{\beta}[0]$ in Δ_{ij} by 1, and use it as the value of $\hat{i}$, by assuming the true value $\vec{\beta}[0]$ is 0.

Similarly, we determine $\hat{j}$ by the last $w_c^{(k)}$ consecutive non-zero or NaN values in $\Delta_{ij}\hat{A}^{(k)}$. Only the correct $(\hat{i}, \hat{j})$ will strictly conform to the matching principles, otherwise we discard the PSP and choose a new one. After recover the pair of $(\hat{i}, \hat{j})$, we construct the Eq. (25). We need slightly more than $h_c^{(k)} w_c^{(k)}$ PSPs to build a full-rank system of Eq. (25) to recover $\vec{C}^{(k)}$ as $\hat{\vec{C}}^{(k)}$.

5.3 Sign Recovery

Reconstruct the convolutional matrix $\hat{A}^{(k)}$ from the recovered kernel weights $\hat{\vec{C}}^{(k)}$. And once extract $\hat{i}$ and $\hat{j}$, the involved LRF-P Ω_t ($0 \leq t < d^{(k+1)}$) corresponding to the PSP could be deduced. According to the prop. 2, the output of i -th and j -th neuron should be the largest value in Ω_t . Then, with $\hat{A}^{(k)}$,

- if $\hat{A}_i^{(k)}(F^{k-1}X + B^{k-1}) = \hat{A}_j^{(k)}(F^{k-1}X + B^{k-1}) > \hat{A}_\ell^{(k)}(F^{k-1}X + B^{k-1})$, $\ell \in \Omega_t$, the sign of the extracted kernel matrix is correct.
- if $\hat{A}_i^{(k)}(F^{k-1}X + B^{k-1}) = \hat{A}_j^{(k)}(F^{k-1}X + B^{k-1}) < \hat{A}_\ell^{(k)}(F^{k-1}X + B^{k-1})$, $\ell \in \Omega_t$, the extracted kernel matrix has an inverted sign, and multiply the vector $\hat{A}_i^{(k)}$ by “-1”.

On the bias recovery. Note that the bias term $b^{(k)}$ is eliminated in Eq. (22) on both sides, making it impossible to derive the bias information from PSP points alone. Consequently, we use the method as described in Eq. (21) in Sect. 4 to recover the bias with a single RPCP.

6 Attack in Practice: Unified Model Combining PSP and RPCP

6.1 Experiment on RPCP and PSP Methods

We first experimentally compare the RPCP and PSP methods on a $(2+1)$ -deep CNN with single input and output channel, trained on the MNIST dataset,

- In the 1st round, the input matrix is convolved with a kernel of size $(1, 1, 5, 5)$, stride $s_c^{(1)} = 1$ and padding $pad^{(1)} = 0$, followed by ReLU functions, and then downsampled by a pooling kernel of size 2×2 , with stride $s_p^{(1)} = 2$.
- In the 2nd round, the operations are identical to those in the 1st round.

We record the number of model queries, attack runtime, and error upper bound, as summarized in Table 2 and discussed as follows.

- **Distribution density of different types of critical points.** Among the identified critical points, the number of PSPs significantly exceeds that of RPCPs within the same layer. Experimental results for the second layer show that out of 10,000 collected critical points, only 15 correspond to RPCPs for that layer, whereas PSP identifies 1548 in Table 2. Consequently, PSP achieves model extraction with shorter runtime and fewer model queries.

Table 2: Comparison of RPCP and PSP Methods

Layer/ Models	Architecture $d^{(k)} - d_f^{(k)} - d^{(k+1)}$	Method	Time seconds	Queries	$(\varepsilon, 0)$	$\max \theta - \hat{\theta} $	Number
1st/(2+1)	1024–784–196	RPCP	$2^{5.39}$	$2^{15.10}$	$2^{-12.78}$	$2^{-16.38}$	1400
		PSP	$2^{4.16}$	$2^{12.62}$	$2^{-32.09}$	$2^{-37.10}$	6487
2nd/(2+1)	196–100–25	RPCP	$2^{6.07}$	$2^{15.95}$	$2^{-16.04}$	$2^{-22.07}$	15
		PSP	$2^{4.60}$	$2^{14.49}$	$2^{-31.74}$	$2^{-35.21}$	1548

Number: Counting from 10,000 collected critical points.

- **High precision extraction.** Recall the linear systems of equations for finally solving the convolution kernel in RPCP Eq. (16) for $A_i^{(k)}$ and PSP Eq. (25) for $\vec{C}^{(k)}$. For RPCPs, we must solve for $|A_i^{(k)}| = d^{(k)}$ variables, whereas for PSPs, the system reduces to only $|\vec{C}^{(k)}| = l_c$ variables, and $d^{(k)} \gg l_c$ according to Eq. (5). Therefore, the system for RPCP possesses higher degrees of freedom. Note that although the non-zero values in $A_i^{(k)}$ in Eq. (5) are the elements of $\vec{C}^{(k)}$, the locations of the zero entries in $A_i^{(k)}$ are unknown before solving the linear system Eq. (16).

According to statistical learning theory, the expected prediction error of a least-squares estimator increases linearly with the number of input parameters [17, Section 2.5]. Consequently, estimating the $d^{(k)}$ variables tends to introduce larger variance compared to solving the low-dimensional PSP system. Meanwhile, the method of least squares tends to distribute values across all coefficients [11], which tends to ‘overfit’ the noise by assigning spurious non-zero values to the zero-valued entries (note that many entries of $A_i^{(k)}$ are zeros), further exacerbating the extraction error. Therefore, the solution to the PSP System of Eq. (25) achieves higher precision.

In our practical experiment in Table 2, the PSP method significantly enhances the precision of parameter extraction. It achieves a much lower upper bound on the maximum output error (the term $(\varepsilon, 0)$) and much lower maximum parameters error (the term $\max|\theta - \hat{\theta}|$) than RPCP method with fewer queries and less time.

- **Bias recovery.** A critical limitation remains: Since PSPs do not reveal any information regarding the bias, we need to collect one RPCP to recover one bias, following the procedure in Eq. (21) in Sect. 4.

6.2 Unified Model Combining PSP and RPCP

Suppose we have precisely extracted the CNN up to layer k , *i.e.*, the recovered $\hat{f}^{(1..k-1)}$ is functionally equivalent to the $f^{(1..k-1)}$ of the victim model. As detailed in Sect. 4 and 5, the procedure for computing the row vector $\hat{A}_i^{(k)}$ from RPCP and extracting position i and j of the pooling selection switch from PSP is

similar (both are based on the computation of the second-order differential function). Meanwhile, we rely on the solution itself to distinguish between RPCPs and PSPs. The full attack is to leverage the PSP method in Sect. 5 for high-precision weights solving, and the RPCP method in Sect. 4 for bias and sign recovery. The detailed procedure is described below.

Step 1. Identify Critical Points and Filter Feasible Candidates. Following Carlini *et al.*'s method [7], we identify a collection of critical points by performing a binary search sweep along continuous linear paths, where the output exhibits a non-linear transition. Then filter out those belonging to previous layers. For each candidate point, we reconstruct the transformation matrix F^{k-1} . Let $N_{positive}^{(k-1)}$ denotes the number of positive values in the output of the $(k-1)$ -th layer. We verify whether the point satisfies the condition: $N_{positive}^{(k-1)} \leq \text{rank}(F^{(k-1)})$. According to [28, Section 4.2], points that fail to meet the constraint will result in underdetermined linear systems of Eq. (15) or (30), and therefore will be discarded.

Step 2. Calculate the Signature. For a critical point X (RPCP or PSP or FCP), we sample tiny perturbations δ in different directions, compute the second-order differential function $\mathcal{H}(X; \vec{\delta}) = \mathcal{F}(X + \vec{\delta}) + \mathcal{F}(X - \vec{\delta}) - 2\mathcal{F}(X)$, which is used in both the RPCP method (e.g., Eq. (13)) and the PSP method (e.g., Eq. (28)) to establish linear equation systems, such as Eq. (15) for RPCP or Eq. (30) for PSP. Let this linear system be $(F^{k-1} \vec{\delta}^{(i)})^T \cdot \vec{\text{sol}} = \mathcal{H}(X; \vec{\delta}^{(i)})$, and solve it by the least square method.

Step 3. Distinguish Critical Points and Extract Position Information. If the critical point in Step 2 is a RPCP, the solution derived will be the row vector $A_i^{(k)}$. Its non-zero entries are strictly confined to the LRF-C of the i -th output neuron (with a length of l_c). If the critical point is a PSP, the solution is the difference vector $A_i^{(k)} - A_j^{(k)}$, when two neurons share the same maximum value within a LRF-P. Here, the non-zero entries correspond to the union of the two respective LRF-Cs.

We extracted the vector $\vec{\text{sol}}$ with all non-zero (exclude NaN) values from the calculated row vector $\vec{\text{sol}}$. Let $l_{\vec{\text{sol}}}$ represents the length of $\vec{\text{sol}}$. Since NaNs appear in the solutions, the length of $l_{\vec{\text{sol}}}$ does not strictly correspond to the above scenarios. We use the relations between $l_{\vec{\text{sol}}}$ and l_c to distinguish if the critical point is a RPCP, or PSP, or FCP:

- $l_{\vec{\text{sol}}} \leq l_c$: It may correspond to RPCP or PSP: we first attempt to match the convolution kernel pattern vector $\vec{V}_i$ defined in Eq. (17) following the method given in Sect. 4.2. If no valid result is returned, we treat it as PSP to extract positional information.
- $l_c < l_{\vec{\text{sol}}} \leq 2l_c$: Since in $A_i^{(k)} - A_j^{(k)}$, non-zero values of the i -th and j -th row vector are offset from each other, the number of non-zero values will not

exceed twice l_c . The critical point can be assumed as PSPs and we retrieve the positional information.

- $l_{sol} > 2l_c$: The critical point is a FCP.

If a PSP is misclassified as an RPCP, it will be automatically filtered out during the matching phase in Step 5. This is because the RPCP method attempts to directly recover the convolution kernel weights, yielding an invalid solution that fails the matching criteria.

Step 4. Calculate Precise Solution. We employ the PSPs to compute a high-precision solution. After we gathering adequate PSPs and their corresponding position information (more than the number of kernel parameter), we establish the linear system of equations Eq. (25) and solve it using singular value decomposition (SVD) method to extract the convolution kernel parameter weights.

Step 5. Recover the Biases and Signs. We employ RPCPs to recover the biases and signs. Suppose a solution derived in Step 2 is recognized as RPCP's in Step 3. Then, the solution contains partial signature of the row vector $A_i^{(k)}$ due to the existence of NaN discussed in Sect. 4.2. With the full signature recovered in Step 4, we match it with the partial signature (ignoring the positions of NaN values) derived by the RPCP according to Eq. (5) to recognize the row index i of $A_i^{(k)}$ of the partial signature. Then, recover the bias according to Eq. (21) and the sign by Sect. 4.3.

When RPCPs are not found in Step 1, we propose a *targeted heuristic search strategy for RPCP* with the signature recovered by PSP method in Step 4.

Targeted heuristic search strategy for RPCP. Our objective is to identify an input $X^{(1)}$ that makes a specific neuron i at layer k reach its critical state *i.e.*, $\hat{A}^{(k)}X^{(k)} + \hat{b}^{(k)} = 0$. Given that the input preprocessing usually normalizes the space of $X^{(1)}$ (e.g., via standardization [19,36] or range scaling [34]), the bias term $\hat{b}^{(k)}$ is not required to compensate for large input offsets [21], *i.e.*, the absolute value of the bias $\hat{b}^{(k)}$ typically small. The strategy is as follows:

1. Recover the sign by PSP method in Sect. 5.3. Reconstruct the kernel matrix $\hat{A}^{(k)}$ with the correct signs.
2. Employ a gradient-based optimizer to search the input space. The optimization process is governed by the three primary constraints:
 - There exists a neuron i that the term $\hat{A}_i^{(k)}X^{(k)}$ approaches 0.
 - Within the same LRF-P Ω_t , the values $\hat{A}_j^{(k)}X^{(k)}$, $j \in \Omega_t$, $j \neq i$ must be substantially smaller than 0.
 - As many neurons as possible in other LRF-Ps have positive values.
3. Upon reaching a candidate point $X^{(1)}$, execute a randomized binary search within the local neighborhood of $X^{(1)}$ and query the CNN. It increases the probability of finding the RPCPs than randomly test the input as in Step 1.

7 Attack on LeNet-5 Style Architectures

This section focuses on the practical convolutional neural network adopting the LeNet-5 Style Architecture [26], which was first proposed by LeCun in 1998. We demonstrate the complete process of applying our proposed algebraic attack to extract the model parameters.

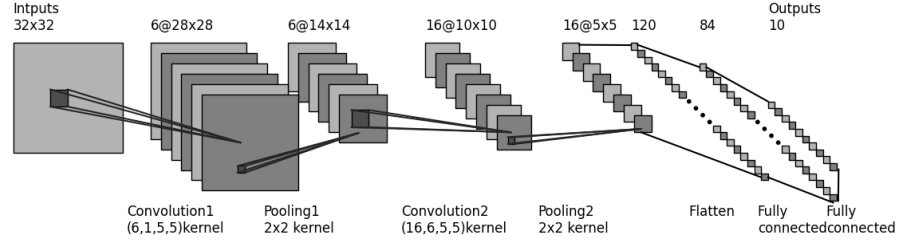


Fig. 7: LeNet-5 Architecture

7.1 LeNet-5 Style Architecture

As shown in Figure 7, the classic LeNet-5 [26] consists of two convolutional rounds and two fully-connected rounds, employing a sparse channel connection in its second convolutional layer, Tanh activation and Average Pooling. However, the sparse structure becomes a computational bottleneck for parallel GPU processing [27,32] and restricts complex feature extraction [14]. Activation functions (Sigmoid/Tanh) pose a higher risk of vanishing and exploding gradients [16] and slow convergence [25]. Consequently, modern implementations of LeNet-5 and other CNNs with similar architecture typically incorporate the following modifications [25,24,36,38]:

- **Convolutional Connectivity:** The original asymmetric sparse connections are replaced with full channel connections. In modern frameworks, the second layer kernel is standardized to a size of $(16, 6, 5, 5)$, with standard convolution stride $s_c^{(k)} = 1$ and pooling stride $s_p^{(k)} = 2$, $(k = 1, 2)$.
- **Activation Function:** The Tanh function is replaced by ReLU.
- **Pooling Strategy:** Average pooling is substituted with Max Pooling to capture the most prominent features.

In this section, our practical attack targets on this modern architecture with raw output accessible. According to our definition, the victim model is a $(2+2)$ -Deep CNN.

7.2 Multiple Channels

LeNet-5 adopts multiple input and output channels in the convolutional block. Suppose in layer k , the convolution layer has $\mathbf{c}_{in}^{(k)}$ input channels and $\mathbf{c}_{out}^{(k)}$ output channels, *i.e.*, there are $\mathbf{c}_{in}^{(k)}$ input matrices, denoted as $IN^{(k),0}, \dots, IN^{(k),\mathbf{c}_{in}^{(k)}-1}$, and $\mathbf{c}_{out}^{(k)}$ output matrices, denoted as $O^{(k),0}, \dots, O^{(k),\mathbf{c}_{out}^{(k)}-1}$. An example of convolutional layer with $\mathbf{c}_{in}^{(k)} = 3$ input channels, $\mathbf{c}_{out}^{(k)} = 2$ output channels is shown in Figure 8. Formally, denote the kernel matrices as $C^{(k),m,n} = [C_{i,j}^{(k),m,n}]_{h_c^{(k)} \times w_c^{(k)}}$,

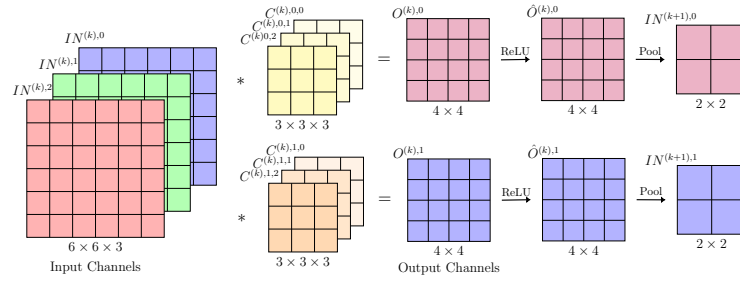


Fig. 8: Multiple Input and Output Channels: $\mathbf{c}_{in}^{(k)} = 3$, $\mathbf{c}_{out}^{(k)} = 2$.

where $0 \leq m \leq \mathbf{c}_{out}^{(k)} - 1$ and $0 \leq n \leq \mathbf{c}_{in}^{(k)} - 1$. The entries $O_{i,j}^{(k),m}$ ($0 \leq m \leq \mathbf{c}_{out}^{(k)} - 1$, $0 \leq i \leq h_o^{(k)} - 1$, $0 \leq j \leq w_o^{(k)} - 1$) of the output matrices generated by the convolution operation are defined as:

$$O_{i,j}^{(k),m} = \sum_{n=0}^{\mathbf{c}_{in}^{(k)}-1} \sum_{p=0}^{h_c^{(k)}-1} \sum_{q=0}^{w_c^{(k)}-1} IN_{i+p,j+q}^{(k),n} C_{p,q}^{(k),m,n} + b^{(k),m}, \quad (31)$$

where $b^{(k),m}$ are the biases for the m -th output channel, and the stride $s_c^{(k)} = 1$.

First Layer of LeNet-5. As shown in Figure 9, the input and output channel of first layer LeNet-5 are $\mathbf{c}_{in}^{(k)} = 1$, $\mathbf{c}_{out}^{(k)} = 6$. The sizes of the output matrices $O^{(1),m}$ ($0 \leq m \leq 5$) are $h_o^{(k)} \times w_o^{(k)} = 5 \times 5$. In vector form, the operation of the first convolutional layer can be modeled as applying six distinct convolution kernel matrices to the input vector $X^{(1)}$. The 6 output matrices are then flattened and sequentially concatenated to form a single vector $Y^{(1)}$ as shown in right side of Figure 9. Consequently, the convolutional matrix $A^{(1)}$ for this layer can be constructed by vertically stacking (or concatenating) the six corresponding convolutional sub-matrices, as shown in Figure 9. More details are given in the following 2nd layer of LeNet-5.

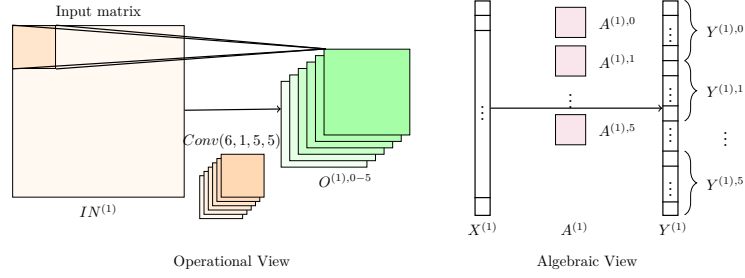


Fig. 9: Operational View and Algebraic View, $(c_{out}^{(k)}, c_{in}^{(k)}, h_c^{(k)}, w_c^{(k)}) = (6, 1, 5, 5)$.

Second Layer of LeNet-5. Similarly, from an algebraic perspective, the operation of the second convolutional layer involves 6 input channels and 16 output channels, *i.e.*, $c_{in}^{(2)} = 6, c_{out}^{(2)} = 16$. The 6 input matrices $IN^{(2),0-5}$ are transformed into a single vector $X^{(2)} = (X^{(2),0}, X^{(2),1}, \dots, X^{(2),n}, \dots, X^{(2),5})^T$, where $X^{(2),n} = (IN_0^{(2),n}, \dots, IN_{h_{in}^{(2)}-1}^{(2),n})^T$. According to Eq. (31), Specifically, $X^{(2)}$ is convolved with $A^{(2),m}$ (the kernel corresponding to the m -th output channel), and the 6 input channels are accumulated element-wise to generate the target 16 feature matrices $O^{(2),m}$ ($0 \leq m \leq 15$). Flattening the feature matrix $O^{(2),m}$, $Y^{(2),m} = (O_0^{(2),m}, \dots, O_{h_o^{(2)}-1}^{(2),m})^T$. Consequently, the single $(d_f^{(2)} = c_{out}^{(2)} \cdot h_o^{(2)} \cdot w_o^{(2)})$ -dimensional output vector $Y^{(2)}$, which is composed of multiple output channels, is denoted as $Y^{(2)} = (Y^{(2),0}, Y^{(2),1}, \dots, Y^{(2),15})^T$, as shown in figure 10.

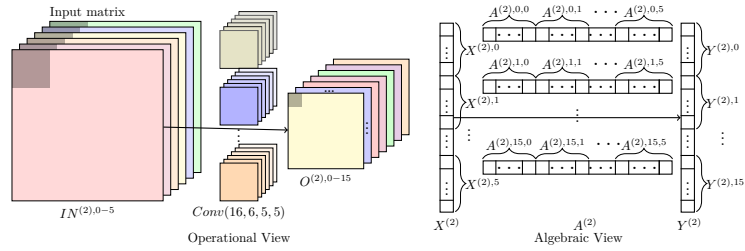


Fig. 10: Operational View and Algebraic View of multiple input and output channels convolutional operations. $(c_{out}^{(k)}, c_{in}^{(k)}, h_c^{(k)}, w_c^{(k)}) = (16, 6, 5, 5)$.

For the i -th output channel, this operation is equivalent to horizontally concatenating 6 individual convolutional matrices into a matrix $A^{(2),i} = (A^{(2),i,0}, A^{(2),i,1}, \dots, A^{(2),i,5})$ ($0 \leq i \leq 15$), where $A^{(2),i,j}$ ($0 \leq j \leq 5$) is derived by Def. 2. For the full output of the second convolutional layer, 16 such matrices $A^{(2),i}$ ($0 \leq i \leq 15$) are vertically stacked to form the final convolutional matrix $A^{(2)}$.

Based on our analysis, we can derive the algebraic expression of the multi-channel convolutional layer: Suppose that for the i -th output channel and the j -th input channel, the convolutional matrix of the corresponding single convolution kernel is $A^{(k),i,j}$ derived by Def. 2, then the full convolutional matrix of the entire k -th convolutional layer is

$$A^{(k)} = \begin{pmatrix} A^{(k),0,0} & A^{(k),0,1} & \dots & A^{(k),0,c_{in}^{(k)}-1} \\ A^{(k),1,0} & A^{(k),1,1} & \dots & A^{(k),1,c_{in}^{(k)}-1} \\ \vdots & \vdots & \ddots & \vdots \\ A^{(k),c_{out}^{(k)}-1,0} & A^{(k),c_{out}^{(k)}-1,1} & \dots & A^{(k),c_{out}^{(k)}-1,c_{in}^{(k)}-1} \end{pmatrix}. \quad (32)$$

Therefore, the multiple channels can also be transformed into the regular operations *i.e.*, the input vector is multiplied with a matrix to derive the output vector. Hence, Def. 5 also holds in this setting, and our algebraic attacks based on the RPCP or PSP method proposed in Sect. 4 and 5 also works.

8 End-to-End Practical Experiments

Taken the full CNN $\mathcal{F}_\theta$ as a black box, with the inputs and the raw outputs $\mathcal{F}_\theta$, we implemented the algebraic attacks on a variety of CNNs. Most experiments were conducted on a server equipped with an NVIDIA RTX 5080 GPU (16 GB memory), although the attack can also be executed on CPU.

Extraction the parameters of the convolutional block of CNNs. Given a $(m+n)$ -deep CNN’s inputs and outputs, we first perform the experiment of the algebraic attack to recover the parameters of the m -round *Convolutional Block*, which directly proves the efficiency and correctness of our attack. The experiments are performed on various CNNs with different architectures (including the modern implementation of LeNet-5) with details given in **Supplementary Material A.2**, whose parameters were trained on random data, MNIST or CIFAR 10, respectively. The results of the experiments are summarized in the top 5 experiments in Table 1 in Sect. 1.

End-to-end Attack on Full and Reduced LeNet-5. Combining Carlini *et al.*’s differential attack [7] and Canales-Martínez *et al.*’s SOE sign-recovery method and Neuron Wiggle sign-recovery method [5], we perform a full parameter extraction on both the m -round *Convolutional Block* and n -round *Fully-connected Block*, as well as the final output layer. The experiment is performed on a reduced version of the modern $(2+2)$ -deep LeNet-5. It is $(2+1)$ -deep CNN with the same 2-round *Convolutional Block* as the full LeNet-5, and 1-round *Fully-connected Block* with the number of neurons reduced from 84 to 20, as well as a final output layer.

We also attack the full modern LeNet-5, extending black-box extraction to the convolutional block and FCNN Round 1. If the input distribution of random

binary search yields samples that only fall into activated (inactivated) region of a neuron, the neuron is labeled as a persistent (dead) neuron, according to [22]. Due to the existence of the persistent and dead neurons, we recover 93.3% of the weights in FCNN Round 1. However, a functionally equivalent attack on the FCNN block could be implemented with the CrossLayer method in [22].

The details of the reduced and full LeNet-5 are given in [Supplementary Material A.2](#). The experiment results and the comparison with Carlini *et al.*'s experiment [7] and Foerster *et al.*'s experiment [13] are given in the last four rows of Table 1 in Sect. 1. A more detailed results table, including runtime, is provided in Table 3 in [Supplementary Material A.1](#).

9 Discussions and Conclusions

With the development of CNN, many other structural hyperparameters have emerged, such as zero-padding [25,36,38,19], various convolutional strides $s_c > 1$ [25,38,19], various pooling strides s_p , or dropout technique [25,36,38]. We discuss their effects on our attacks in [Supplementary Material B](#).

In this paper, we tackle the intractable problem of cryptanalytic parameter extraction on Convolutional Neural Networks (CNNs) with max pooling for the first time. Different from existing works focusing only on ReLU-based fully connected neural networks (FCNNs), we extend cryptanalytic extraction attacks to CNNs by overcoming max pooling-induced nonlinearity challenges. We propose an algebraic representation of CNNs, identify two novel critical points (RPCPs and PSPs), and design corresponding extraction techniques. A unified model integrating RPCPs and PSPs is developed to leverage their strengths, with a heuristic strategy to address RPCP rarity. End-to-end black-box experiments on multiple CNNs validate our method's effectiveness in recovering full CNN parameters. Future work will extend the method to hard-label scenarios and complex CNN architectures.

References

1. Lejla Batina, Shivam Bhasin, Dirmanto Jap, and Stjepan Picek. {CSI}{NN}: Reverse engineering of neural network architectures through electromagnetic side channel. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 515–532, 2019.
2. Avrim Blum and Ronald Rivest. Training a 3-node neural network is np-complete. *Advances in neural information processing systems*, 1, 1988.
3. Y-Lan Boureau, Jean Ponce, and Yann LeCun. A theoretical analysis of feature pooling in visual recognition. In *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, pages 111–118, 2010.
4. Isaac A. Canales-Martínez and David Santos. Extracting some layers of deep neural networks in the hard-label setting. In Daniel Escudero and Ivan Damgård, editors, *Progress in Cryptology - LATINCRYPT 2025 - 9th International Conference on Cryptology and Information Security in Latin America, Medellín, Colombia, October 1-3, 2025, Proceedings*, volume 16129 of *Lecture Notes in Computer Science*, pages 399–421. Springer, 2025.

5. Isaac Andrés Canales-Martínez, Jorge Chávez-Saab, Anna Hambitzer, Francisco Rodríguez-Henríquez, Nitin Satpute, and Adi Shamir. Polynomial time cryptanalytic extraction of neural network models. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part III*, volume 14653 of *Lecture Notes in Computer Science*, pages 3–33. Springer, 2024.
6. Nicholas Carlini, Jorge Chávez-Saab, Anna Hambitzer, Francisco Rodríguez-Henríquez, and Adi Shamir. Polynomial time cryptanalytic extraction of deep neural networks in the hard-label setting. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part I*, volume 15601 of *Lecture Notes in Computer Science*, pages 364–396. Springer, 2025.
7. Nicholas Carlini, Matthew Jagielski, and Ilya Mironov. Cryptanalytic extraction of neural network models. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part III*, volume 12172 of *Lecture Notes in Computer Science*, pages 189–218. Springer, 2020.
8. Yi Chen, Xiaoyang Dong, Jian Guo, Yantian Shen, Anyu Wang, and Xiaoyun Wang. Hard-label cryptanalytic extraction of neural network models. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VIII*, volume 15491 of *Lecture Notes in Computer Science*, pages 207–236. Springer, 2024.
9. Yi Chen, Xiaoyang Dong, Ruijie Ma, Yantian Shen, Anyu Wang, Hongbo Yu, and Xiaoyun Wang. Delving into cryptanalytic extraction of prelu neural networks. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 576–607. Springer, 2025.
10. Itai Dinur, Orr Dunkelman, and Adi Shamir. Collision attacks on up to 5 rounds of SHA-3 using generalized internal differentials. In Shiho Moriai, editor, *Fast Software Encryption - 20th International Workshop, FSE 2013, Singapore, March 11-13, 2013. Revised Selected Papers*, volume 8424 of *Lecture Notes in Computer Science*, pages 219–240. Springer, 2013.
11. D.L. Donoho. Compressed sensing. *IEEE Transactions on Information Theory*, 52(4):1289–1306, 2006.
12. Charles Fefferman et al. Reconstructing a neural net from its output. *Revista Matemática Iberoamericana*, 10(3):507–556, 1994.
13. Hanna Foerster, Robert Mullins, Ilia Shumailov, and Jamie Hayes. Beyond slow signs in high-fidelity model extraction. *Advances in Neural Information Processing Systems*, 37:19496–19522, 2024.
14. Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019.
15. Kunihiko Fukushima, Sei Miyake, and Takayuki Ito. Neocognitron: A neural network model for a mechanism of visual pattern recognition. *IEEE transactions on systems, man, and cybernetics*, (5):826–834, 1983.

16. Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In Yee Whye Teh and D. Mike Titterton, editors, *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2010, Chia Laguna Resort, Sardinia, Italy, May 13-15, 2010*, volume 9 of *JMLR Proceedings*, pages 249–256. JMLR.org, 2010.
17. Trevor Hastie, Robert Tibshirani, and Jerome H. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd Edition*. Springer Series in Statistics. Springer, 2009.
18. Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.
19. Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
20. Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4700–4708, 2017.
21. Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Francis R. Bach and David M. Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, volume 37 of *JMLR Workshop and Conference Proceedings*, pages 448–456. JMLR.org, 2015.
22. Akira Ito, Takayuki Miura, and Yosuke Todo. Is the hard-label cryptanalytic model extraction really polynomial? *arXiv preprint arXiv:2510.06692*, 2025.
23. Matthew Jagielski, Nicholas Carlini, David Berthelot, Alex Kurakin, and Nicolas Papernot. High accuracy and high fidelity extraction of neural networks. In *29th USENIX security symposium (USENIX Security 20)*, pages 1345–1362, 2020.
24. Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross B. Girshick, Sergio Guadarrama, and Trevor Darrell. Caffe: Convolutional architecture for fast feature embedding. In Kien A. Hua, Yong Rui, Ralf Steinmetz, Alan Hanjalic, Apostol Natsev, and Wenwu Zhu, editors, *Proceedings of the ACM International Conference on Multimedia, MM '14, Orlando, FL, USA, November 03 - 07, 2014*, pages 675–678. ACM, 2014.
25. Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
26. Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 2002.
27. Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. Pruning filters for efficient convnets. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.
28. Haolin Liu, Adrien Siproudhis, Samuel Experton, Peter Lorenz, Christina Boura, and Thomas Peyrin. Navigating the deep: Signature extraction on deep neural networks. *arXiv preprint arXiv:2506.17047*, 2025.
29. Daniel Lowd and Christopher Meek. Adversarial learning. In *Proceedings of the eleventh ACM SIGKDD international conference on Knowledge discovery in data mining*, pages 641–647, 2005.

30. Kyle Matoba, Nikolaos Dimitriadis, and François Fleuret. Benefits of max pooling in neural networks: Theoretical and experimental evidence. *Transactions on Machine Learning Research*, 2023.
31. Daryna Oliynyk, Rudolf Mayer, and Andreas Rauber. I know what you trained last summer: A survey on stealing machine learning models and defences. *ACM Computing Surveys*, 55(14s):1–41, 2023.
32. Sang-Soo Park, Jung-Hyun Hong, and Ki-Seok Chung. Modified convolution neural network for highly effective parallel processing. In Chengcui Zhang, Balaji Palanisamy, Latifur Khan, and Sahra Sedigh Sarvestani, editors, *2017 IEEE International Conference on Information Reuse and Integration, IRI 2017, San Diego, CA, USA, August 4-6, 2017*, pages 325–331. IEEE Computer Society, 2017.
33. Thomas Peyrin. Improved differential attacks for ECHO and grøstl. In Tal Rabin, editor, *Advances in Cryptology - CRYPTO 2010, 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010. Proceedings*, volume 6223 of *Lecture Notes in Computer Science*, pages 370–392. Springer, 2010.
34. Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. In Yoshua Bengio and Yann LeCun, editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016.
35. David Rolnick and Konrad Kording. Reverse-engineering deep relu networks. In *International conference on machine learning*, pages 8178–8187. PMLR, 2020.
36. Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015.
37. Xiaohan Sun, Hao Lei, Longxiang Wei, Xiaokang Qi, Kai Hu, Meiqin Wang, and Wei Wang. Cryptanalytic extraction of convolutional neural networks. *Cryptology ePrint Archive*, Paper 2026/139, 2026.
38. Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9, 2015.
39. Florian Tramèr, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Stealing machine learning models via prediction {APIs}. In *25th USENIX security symposium (USENIX Security 16)*, pages 601–618, 2016.
40. Du Tran, Lubomir Bourdev, Rob Fergus, Lorenzo Torresani, and Manohar Paluri. Learning spatiotemporal features with 3d convolutional networks. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 4489–4497. IEEE, 2015.
41. Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.
42. Xiang Zhang, Junbo Jake Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, pages 649–657, 2015.

Supplementary Material

A More Detailed Information in Experiments

In this section, we demonstrate the full details of our experimental results in Sect. A.1 and describe the detailed structure information of models we experimented on in Sect. A.2.

A.1 Experiment Results

We include a more concise version of our experimental results in Section 1 (*i.e.*, Table 1). For the complete experimental result metrics – including runtime, model queries, and error evaluation $(\varepsilon, 0)$ and $\max|\theta - \hat{\theta}|$ – we provide these full details in Table 3.

Table 3: Experiments on different CNNs.

Models ($m + n$)	Trainset	Architecture $d^{(k)} - d_f^{(k)} - d^{(k+1)}$	Kernel [†]	Param	Runtime Seconds	Queries	$(\varepsilon, 0)$ [‡]	$\max \theta - \hat{\theta} $ [‡]
(1+1)	Random	Conv R1: 64–36–9	(1,1,3,3)	10	$2^{-0.26}$	$2^{9.98}$	$2^{-43.19}$	$2^{-47.52}$
(2+1)	MNIST	Conv R1: 1024–784–196 Conv R2: 196–100–25	(1,1,5,5) (1,1,5,5)	52	$2^{5.97}$	$2^{15.06}$	$2^{-30.20}$	$2^{-34.65}$
(2+2)*	MNIST	Conv R1: 1024–4704–1176 Conv R2: 1176–1600–400	(1,6,5,5) (6,16,5,5)	2572	$2^{17.21}$	$2^{27.29}$	$2^{-23.38}$	$2^{-27.31}$
(3+1) ^{P2}	MNIST	Conv R1: 1024–1024–256 Conv R2: 256–256–64 Conv R3: 64–64–16	(1,1,5,5) (1,1,5,5) (1,1,5,5)	78	$2^{8.76}$	$2^{16.90}$	$2^{-27.54}$	$2^{-31.15}$
(2+2) ^{P1}	CIFAR10	Conv R1: 3072–3072–768 Conv R2: 768–1024–256	(3,3,3,3) (3,4,3,3)	196	$2^{10.96}$	$2^{18.37}$	$2^{-29.13}$	$2^{-36.19}$
(2+1) [♣]	MNIST	Conv R1: 1024–4704–1176 Conv R2: 1176–1600–400 FCNN: 400–20–10	(1,6,5,5) (6,16,5,5) –	10772 (2572+8230)	$2^{17.38}$	$2^{27.65}$	$2^{-22.06}$	$2^{-27.14}$
(2+2)* [▲]	MNIST	C₁ : 1024–4704–1176 C₂ : 1176–1600–400 FCNN: 400–120 –84–10	(1,6,5,5) (6,16,5,5) –	61706 (2572+59134)	$2^{18.46}$	$2^{27.31}$	–	$2^{-26.03}$
(0+3) [7]	MNIST	FCNN: 40–20–10–10–1	–	1110	–	$2^{17.8}$	$2^{-23.4}$	$2^{-27.1}$
(0+3) [13] [♠]	MNIST	FCNN: 784–16– 16–16 –1	–	272	$2^{16.0}$	$2^{25.4}$	–	–

*: LeNet-5 modern model, specified in in Sect. 7.1.

^{P1}, ^{P2}: p_i indicates $pad^{(k)} = i$ defined as “same padding” in Supplementary Material B; otherwise, $pad^{(k)} = 0$.

[†]: Convolutional kernel size of $(c_{in}^{(k)}, c_{out}^{(k)}, h_c^{(k)}, w_c^{(k)})$, with stride $s_c^{(k)} = 1$.

[‡]: The accuracy term $(\varepsilon, 0)$ is defined in Sect. 2.3, $\max|\theta - \hat{\theta}|$ directly measures the maximum extraction error of model parameters.

[♣]: Reduced version of LeNet-5 with only one FCNN round.

[▲]: Red bold layer in Architecture indicates end-to-end extracting the full convolutional block and 93.33% weights of FCNN Round 1 (not full due to existing dead neurons and persistent neurons according to [22]).

[♠]: Red bold layer in Architecture indicates extracting the single layer, with prior layers’ extractions assumed to be correct.

A.2 Detailed Model Structure Information in Experiments

(1+1)-Deep CNN:

1. Convolutional Round 1:
 - **Convolutional layer** $f_c^{(1)}$: 8×8 Input matrix $IN^{(1)}$, convolved with $(1, 1, 3, 3)$ convolution kernel, with stride $s_c^{(1)} = 1$, padding $pad^{(1)} = 0$, produces 6×6 output matrix $O^{(1)}$, followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(1)}$: 2×2 max pooling kernel, with stride $s_\rho^{(1)} = 2$, produces 3×3 output matrix $IN^{(2)}$.
2. FCNN Round 1:
 - Flatten the matrix $IN^{(2)}$ and get the 9-dimensional vector $X^{(1)}$.
 - **Fully-connected Layer** $f^{(1)}$: $d^{(1)} = 9$ input neurons and $d^{(2)} = 5$ output neurons, followed by ReLU activation.
3. Last Layer $f^{(2)}$: $d^{(2)} = 5$ input neurons and $d^{(3)} = 10$ output neurons.

(2+1)-Deep CNN:

1. Convolutional Round 1:
 - **Convolutional layer** $f_c^{(1)}$: 32×32 Input matrix $IN^{(1)}$, convolved with $(1, 1, 5, 5)$ convolution kernel, with stride $s_c^{(1)} = 1$, padding $pad^{(1)} = 0$, produces 28×28 output matrix $O^{(1)}$, followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(1)}$: 2×2 max pooling kernel, with stride $s_\rho^{(1)} = 2$, produces 14×14 output matrix $IN^{(2)}$.
2. Convolutional Round 2:
 - **Convolutional layer** $f_c^{(2)}$: 14×14 Input matrix $IN^{(2)}$, convolved with $(1, 1, 5, 5)$ convolution kernel, with stride $s_c^{(2)} = 1$, padding $pad^{(2)} = 0$, produces 10×10 output matrix $O^{(2)}$, followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(2)}$: 2×2 max pooling kernel, with stride $s_\rho^{(2)} = 2$, produces 5×5 output matrix $IN^{(3)}$.
3. FCNN Round 1:
 - Flatten the matrix $IN^{(3)}$ and get the 25-dimensional vector $X^{(1)}$.
 - **Fully-connected Layer** $f^{(1)}$: $d^{(1)} = 25$ input neurons and $d^{(2)} = 18$ output neurons, followed by ReLU activation.
4. Last Layer $f^{(2)}$: $d^{(2)} = 18$ input neurons and $d^{(3)} = 10$ output neurons.

(2+2)-Deep LeNet-5:

1. Convolutional Round 1:
 - **Convolutional layer** $f_c^{(1)}$: 32×32 Input matrix $IN^{(1)}$, convolved with $(6, 1, 5, 5)$ convolution kernel, with stride $s_c^{(1)} = 1$, padding $pad^{(1)} = 0$, produces 6 output matrix $O^{(1)}$ of size 28×28 , followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(1)}$: 2×2 max pooling kernel, with stride $s_\rho^{(1)} = 2$, produces 6 output matrix $IN^{(2)}$ of size 14×14 .
2. Convolutional Round 2:

- **Convolutional layer** $f_c^{(2)}$: 6 Input matrix $IN^{(2)}$ of size 14×14 , convolved with $(6, 16, 5, 5)$ convolution kernel, with stride $s_c^{(2)} = 1$, padding $pad^{(2)} = 0$, produces 16 output matrix $O^{(2)}$ of size 10×10 , followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(2)}$: 2×2 max pooling kernel, with stride $s_\rho^{(2)} = 2$, produces 16 output matrix $IN^{(3)}$ of size 5×5 .
3. FCNN Round 1:
 - Flatten the matrix $IN^{(3)}$ and get the 400-dimensional vector $X^{(1)}$.
 - **Fully-connected Layer** $f^{(1)}$: $d^{(1)} = 400$ input neurons and $d^{(2)} = 120$ output neurons, followed by ReLU activation.
 4. FCNN Round 2:
 - **Fully-connected Layer** $f^{(2)}$: $d^{(2)} = 120$ input neurons and $d^{(3)} = 84$ output neurons, followed by ReLU activation.
 5. Last Layer $f^{(3)}$: $d^{(3)} = 84$ input neurons and $d^{(3)} = 10$ output neurons.

(3+1)-Deep CNN:

1. Convolutional Round 1:
 - **Convolutional layer** $f_c^{(1)}$: 32×32 Input matrix $IN^{(1)}$, convolved with $(1, 1, 5, 5)$ convolution kernel, with stride $s_c^{(1)} = 1$, padding $pad^{(1)} = 2$, produces 32×32 output matrix $O^{(1)}$, followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(1)}$: 2×2 max pooling kernel, with stride $s_\rho^{(1)} = 2$, produces 16×16 output matrix $IN^{(2)}$.
2. Convolutional Round 2:
 - **Convolutional layer** $f_c^{(2)}$: 16×16 Input matrix $IN^{(2)}$, convolved with $(1, 1, 5, 5)$ convolution kernel, with stride $s_c^{(2)} = 1$, padding $pad^{(2)} = 2$, produces 16×16 output matrix $O^{(2)}$, followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(2)}$: 2×2 max pooling kernel, with stride $s_\rho^{(2)} = 2$, produces 8×8 output matrix $IN^{(3)}$.
3. Convolutional Round 3:
 - **Convolutional layer** $f_c^{(3)}$: 8×8 Input matrix $IN^{(3)}$, convolved with $(1, 1, 5, 5)$ convolution kernel, with stride $s_c^{(3)} = 1$, padding $pad^{(3)} = 2$, produces 8×8 output matrix $O^{(3)}$, followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(3)}$: 2×2 max pooling kernel, with stride $s_\rho^{(3)} = 2$, produces 4×4 output matrix $IN^{(4)}$.
4. FCNN Round 1:
 - Flatten the matrix $IN^{(4)}$ and get the 16-dimensional vector $X^{(1)}$.
 - **Fully-connected Layer** $f^{(1)}$: $d^{(1)} = 16$ input neurons and $d^{(2)} = 25$ output neurons, followed by ReLU activation.
5. Last Layer $f^{(2)}$: $d^{(2)} = 25$ input neurons and $d^{(3)} = 10$ output neurons.

(2+2)-Deep CNN trained on CIFAR 10:

1. Convolutional Round 1:
 - **Convolutional layer** $f_c^{(1)}$: 3 Input matrix $IN^{(1)}$ of size 32×32 , convolved with $(1, 1, 3, 3)$ convolution kernel, with stride $s_c^{(1)} = 1$, padding $pad^{(1)} = 1$, produces 3 output matrix $O^{(1)}$ of size 32×32 , followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(1)}$: 2×2 max pooling kernel, with stride $s_\rho^{(1)} = 2$, produces 3 output matrix $IN^{(2)}$ of size 16×16 .
2. Convolutional Round 2:
 - **Convolutional layer** $f_c^{(2)}$: 3 Input matrix $IN^{(2)}$ of size 16×16 , convolved with $(3, 4, 3, 3)$ convolution kernel, with stride $s_c^{(2)} = 1$, padding $pad^{(2)} = 1$, produces 4 output matrix $O^{(2)}$ of size 16×16 , followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(2)}$: 2×2 max pooling kernel, with stride $s_\rho^{(2)} = 2$, produces 4 output matrix $IN^{(3)}$ of size 8×8 .
3. FCNN Round 1:
 - Flatten the matrix $IN^{(3)}$ and get the 256-dimensional vector $X^{(1)}$.
 - **Fully-connected Layer** $f^{(1)}$: $d^{(1)} = 256$ input neurons and $d^{(2)} = 256$ output neurons, followed by ReLU activation.
4. FCNN Round 2:
 - **Fully-connected Layer** $f^{(2)}$: $d^{(2)} = 256$ input neurons and $d^{(3)} = 120$ output neurons, followed by ReLU activation.
5. Last Layer $f^{(3)}$: $d^{(3)} = 120$ input neurons and $d^{(3)} = 10$ output neurons.

(2+1)-Deep LeNet-5:

1. Convolutional Round 1:
 - **Convolutional layer** $f_c^{(1)}$: 32×32 Input matrix $IN^{(1)}$, convolved with $(6, 1, 5, 5)$ convolution kernel, with stride $s_c^{(1)} = 1$, padding $pad^{(1)} = 0$, produces 6 output matrix $O^{(1)}$ of size 28×28 , followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(1)}$: 2×2 max pooling kernel, with stride $s_\rho^{(1)} = 2$, produces 6 output matrix $IN^{(2)}$ of size 14×14 .
2. Convolutional Round 2:
 - **Convolutional layer** $f_c^{(2)}$: 6 Input matrices $IN^{(2)}$ of size 14×14 , convolved with $(6, 16, 5, 5)$ convolution kernel, with stride $s_c^{(2)} = 1$, padding $pad^{(2)} = 0$, produces 16 output matrices $O^{(2)}$ of size 10×10 , followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(2)}$: 2×2 max pooling kernel, with stride $s_\rho^{(2)} = 2$, produces 16 output matrix $IN^{(3)}$ of size 5×5 .
3. FCNN Round 1:
 - Flatten the matrix $IN^{(3)}$ and get the 400-dimensional vector $X^{(1)}$.
 - **Fully-connected Layer** $f^{(1)}$: $d^{(1)} = 400$ input neurons and $d^{(2)} = 20$ output neurons, followed by ReLU activation.
4. Last Layer $f^{(2)}$: $d^{(2)} = 20$ input neurons and $d^{(3)} = 10$ output neurons.

Modern (2+2)-Deep LeNet-5:

1. Convolutional Round 1:
 - **Convolutional layer** $f_c^{(1)}$: 32×32 Input matrix $IN^{(1)}$, convolved with $(6, 1, 5, 5)$ convolution kernel, with stride $s_c^{(1)} = 1$, padding $pad^{(1)} = 0$, produces 6 output matrix $O^{(1)}$ of size 28×28 , followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(1)}$: 2×2 max pooling kernel, with stride $s_\rho^{(1)} = 2$, produces 6 output matrix $IN^{(2)}$ of size 14×14 .
2. Convolutional Round 2:
 - **Convolutional layer** $f_c^{(2)}$: 6 Input matrices $IN^{(2)}$ of size 14×14 , convolved with $(6, 16, 5, 5)$ convolution kernel, with stride $s_c^{(2)} = 1$, padding $pad^{(2)} = 0$, produces 16 output matrices $O^{(2)}$ of size 10×10 , followed by ReLU activation.
 - **Max Pooling layer** $\rho^{(2)}$: 2×2 max pooling kernel, with stride $s_\rho^{(2)} = 2$, produces 16 output matrix $IN^{(3)}$ of size 5×5 .
3. FCNN Round 1:
 - Flatten the matrix $IN^{(3)}$ and get the 400-dimensional vector $X^{(1)}$.
 - **Fully-connected Layer** $f^{(1)}$: $d^{(1)} = 400$ input neurons and $d^{(2)} = 120$ output neurons, followed by ReLU activation.
4. FCNN Round 2:
 - **Fully-connected Layer** $f^{(2)}$: $d^{(2)} = 120$ input neurons and $d^{(3)} = 84$ output neurons, followed by ReLU activation.
5. Last Layer $f^{(3)}$: $d^{(3)} = 84$ input neurons and $d^{(4)} = 10$ output neurons.

B Analysis of Other Structural Hyperparameters

In this section, we discuss some variants of CNN. Specifically, we focus on modifications to those structural hyperparameters, including padding, stride (in both convolutional and pooling layers), and dropout mechanisms. Naturally, this raises a question: *Will our attack still succeed with the modifications to the hyperparameters?*

B.1 Zero-padding in the Convolutional Layer

To fully capture the features near the boundaries, zeros are usually padded around the input matrix $IN^{(k)}$. For example, $pad^{(k)} = 1$ denotes adding one row/column of zeros to all four sides of $IN^{(k)}$. Consequently, the output matrix $O^{(k)}$ after convolution with $pad^{(k)}$ has a shape of $h_o^{(k)} \times w_o^{(k)}$, where

$$h_o^{(k)} = \left\lfloor \frac{h_{in}^{(k)} + 2 \cdot pad^{(k)} - h_c^{(k)}}{s_c^{(k)}} \right\rfloor + 1, \quad w_o^{(k)} = \left\lfloor \frac{w_{in}^{(k)} + 2 \cdot pad^{(k)} - w_c^{(k)}}{s_c^{(k)}} \right\rfloor + 1 \quad (33)$$

Two common types of padding are widely used in practice:

1. **Valid padding.** No zero padding is applied ($pad^{(k)} = 0$). The convolution kernel is restricted to visit only positions where it is fully contained within the input boundaries.
2. **Same padding.** Zeros are padded around the input matrix such that the size of the output feature map is identical to those of the input.

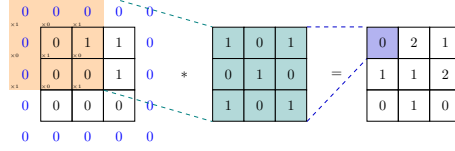


Fig. 11: Example of Convolution Operation with zero padding

Figure 11 illustrates a scenario using same padding, with $c_{in}^{(k)} = c_{out}^{(k)} = s_c^{(k)} = 1$, where the input matrix $IN^{(k)}$ has dimensions $h_{in}^{(k)} \times w_{in}^{(k)} = 3 \times 3$, i.e., $d^{(k)} = 9$. Given a $h_c^{(k)} \times w_c^{(k)} = 3 \times 3$ kernel matrix $C^{(k)}$, with the padding $pad^{(k)} = 1$, the output matrix $O^{(k)}$ is of dimension $h_o^{(k)} \times w_o^{(k)} = 3 \times 3$, i.e., $d_f^{(k)} = 9$. In the algebraic view, the padding operation can be modeled as padding zeros to the input vector $X^{(k)}$ and get the zero-padded input vector $X_{pad}^{(k)} \in \mathbb{R}^{d_{pad}^{(k)}}$, $d_{pad}^{(k)} = (h_{in}^{(k)} + 2 \cdot pad^{(k)}) \cdot (w_{in}^{(k)} + 2 \cdot pad^{(k)})$, i.e., $d_{pad}^{(k)} = 25$. According the Def. 2, the convolutional matrix corresponding to $X_{pad}^{(k)}$ is:

$$A_{pad}^{(k)} = \begin{pmatrix} c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 & 0 & 0 \end{pmatrix}.$$

However, for the padded input vector $X_{pad}^{(k)} \in \mathbb{R}^{d_{pad}^{(k)}}$, we can only change the $d^{(k)}$ dimensions ($d^{(k)} < d_{pad}^{(k)}$), as the padded entries must remain fixed at zero. This constraint results in a loss of full control over the entire input space $\mathbb{R}^{d_{pad}^{(k)}}$. Instead, we continue to use the raw input vector $X^{(k)} \in \mathbb{R}^{d^{(k)}}$, the convolutional matrix $A^{(k)}$ is obtained by extracting the columns of $A_{pad}^{(k)}$ that correspond to the elements of the raw input vector $X^{(k)}$ before padding. Let $Col^{(k)} \subset \{0, 1, \dots, d_{pad}^{(k)}\}$ denotes the set of corresponding column indices, then

$$Col^{(k)} := \{(w_{in}^{(k)} + 2pad^{(k)}) \cdot i + j, pad^{(k)} \leq i \leq pad^{(k)} + h_{in}^{(k)} - 1, pad^{(k)} \leq j \leq pad^{(k)} + w_{in}^{(k)} - 1\}. \quad (34)$$

In this example, the column indices $Col^{(k)} = \{6, \dots, 8, 11, \dots, 13, 16, \dots, 18\}$,

$$A^{(k)} = \begin{pmatrix} c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 & 0 & 0 & 0 \\ c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 & 0 & 0 \\ 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} & 0 & 0 & 0 \\ c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 & c_{2,1}^{(k)} & c_{2,2}^{(k)} & 0 \\ c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} & c_{2,0}^{(k)} & c_{2,1}^{(k)} & c_{2,2}^{(k)} \\ 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} & 0 & c_{2,0}^{(k)} & c_{2,1}^{(k)} \\ 0 & 0 & 0 & c_{0,1}^{(k)} & c_{0,2}^{(k)} & 0 & c_{1,1}^{(k)} & c_{1,2}^{(k)} & 0 \\ 0 & 0 & 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & c_{0,2}^{(k)} & c_{1,0}^{(k)} & c_{1,1}^{(k)} & c_{1,2}^{(k)} \\ 0 & 0 & 0 & 0 & c_{0,0}^{(k)} & c_{0,1}^{(k)} & 0 & c_{1,0}^{(k)} & c_{1,1}^{(k)} \end{pmatrix}.$$

In our attack, the RPCP and PSP method are designed for critical points corresponding to vector $A_i^{(k)}$ and $A_i^{(k)} - A_j^{(k)}$, respectively. Both $A_i^{(k)}$ and $A_j^{(k)}$ need to contain the full information about the kernel matrix (e.g., $A_4^{(k)}$ in this case). Other row vectors with truncated subset of the kernel weights (e.g., $A_0^{(k)} \sim A_3^{(k)}$ and $A_5^{(k)} \sim A_8^{(k)}$ in this case) will interfere with the merging of partial weights in Sect. 4.2 and the computation of $\vec{C}^{(k)}$ in Sect. 5.1.

However, the pattern matching method for RPCPs in Sect. 4.2 are designed to identify the row vector with full kernel parameters, this will filter out noisy RPCPs. For PSPs, after extracting i and j from $\Delta_{ij}\hat{A}^{(k)} = A_i^{(k)} - A_j^{(k)}$ in Sect. 5.2, we can deduce $\mathcal{K}_i$ and $\mathcal{K}_j$ which denotes the set of indices belonging to the LRF-C of the i -th and j -th neuron, respectively, according to Eq. 23. Denote the first element in $\mathcal{K}_i$ as $\mathcal{K}_i[0]$, and the last element in $\mathcal{K}_j$ as $\mathcal{K}_j[l_c^{(k)} - 1]$, where $l_c^{(k)} = h_c^{(k)}w_c^{(k)}$. Appropriate i and j follows the principle: $\frac{\Delta_{ij}\hat{A}^{(k)}[\mathcal{K}_i[0]]}{\Delta_{ij}\hat{A}^{(k)}[\mathcal{K}_j[l_c^{(k)} - 1]]} = \frac{\vec{C}^{(k)}[0]}{\vec{C}^{(k)}[l_c^{(k)} - 1]}$. This enables us to cluster the $\frac{\Delta_{ij}\hat{A}^{(k)}[\mathcal{K}_i[0]]}{\Delta_{ij}\hat{A}^{(k)}[\mathcal{K}_j[l_c^{(k)} - 1]]}$ from each PSP and discard noisy PSPs (if $\Delta_{ij}\hat{A}^{(k)}[\mathcal{K}_i[0]] = \text{NaN}$ or $\Delta_{ij}\hat{A}^{(k)}[\mathcal{K}_j[l_c^{(k)} - 1]] = \text{NaN}$, also discard it).

A small fraction of the differences $A_i^{(k)} - A_j^{(k)}$ from PSPs may yield row vectors structurally identical to the $A_i^{(k)}$ in RPCP method which satisfy the matching principle in Sect. 4.2 and therefore identified as RPCPs. We filter these out via Step 5 in Sect. 6.2. Therefore we can effectively discard these false-positives cases, excluding them from the set of valid information used for reconstruction.

B.2 Stride in Convolutional Layer

If the convolution stride $s_c > 1$, the convolution kernel will skip certain positions when sliding over the input matrix, which can be regarded as downsampling the output of the full convolution ($s_c = 1$). However, this does not undermine our method. While it results in a lower density of available critical points, the

algebraic integrity of the extracted row vector (specifically, $A_i^{(k)}$ derived from RPCPs in Sect. 4.1 and $A_i^{(k)} - A_j^{(k)}$ derived from PSPs in Sect. 5.2) remains complete and valid.

B.3 Stride in Pooling Layer

Different values of the pooling stride $s_\rho^{(k)}$ lead to two distinct scenarios:

1. **Disjoint Pooling** ($s_\rho^{(k)} \geq h_m^{(k)}, w_m^{(k)}$). Some elements of the output $\hat{O}^{(k)}$ after convolution and activation may not be covered by the pooling window.
2. **Overlapping Pooling** ($s_\rho^{(k)} < h_m^{(k)}, w_m^{(k)}$). Pooling windows intersect, meaning a single neuron may belong to several local receptive fields.

Under Disjoint Pooling, when $s_\rho^{(k)} > h_m^{(k)}, w_m^{(k)}$, the pooling kernel skips certain neurons. Note that neuron i in the convolutional layer selects the i -th column of the pooling matrix $P^{(k)}$. If neuron i shows the maximum value in the t -th LRF-P, then we have $P_{t,i}^{(k)} = 1$, otherwise $P_{t,i}^{(k)} = 0$. Therefore, these skipped neurons correspond to constant all-zero column vectors. However, this does not interfere with the identification and calculation of RPCPs and PSPs, as they will not be detected.

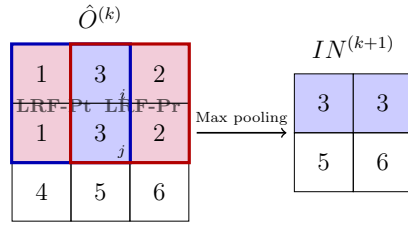


Fig. 12: Overlapping Pooling case 1

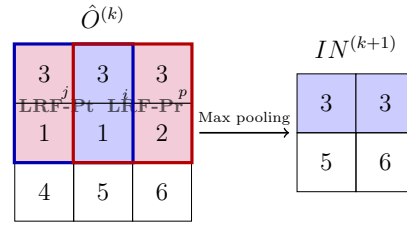


Fig. 13: Overlapping Pooling case 2

Under Overlapping Pooling, for RPCPs, a neuron in the critical state (*i.e.*, input=0) may show the maximum value across multiple overlapping pooling windows, resulting in multiple ones in its column vector. Any pooling window that selects this critical neuron, is sufficient to compute the row vector $A_i^{(k)}$. The simultaneous selection by multiple windows does not induce calculation failure. In the PSP method, suppose in the t -th LRF-P, neuron i and j achieve the same maximum value, as shown in the top-left of Fig. 12 and Fig. 13 (outlined in blue).

1. As illustrated in figure 12, if both neuron i and j also show the maximum value in another r -th LRF-P (outlined in red), the output difference still presents the gradient information $A_i^{(k)} - A_j^{(k)}$.

2. As illustrated in figure 13, if neuron i (j) and another neuron p show the maximum value in r -th LRF-P (outlined in red), with high probability we will get a combination of $A_i^{(k)} - A_j^{(k)}$ and $A_i^{(k)} - A_p^{(k)}$ ($A_j^{(k)} - A_p^{(k)}$). However, the extraction in Sect. 5.2 will return no valid result and therefore filter out these noises.

B.4 Dropout

The dropout technique in CNN was first introduced by Krizhevsky *et al.* [25] in 2012, working by setting the output of each hidden neuron zero with a predefined probability during the training period. The neurons which are “dropped out” in this way do not contribute to the forward and backward procedure. However, this technique does not delete neurons from the network topology. The model function f is still static and deterministic. From a practical attack perspective, this mechanism makes no difference to our attack.